



Cloud Computing Assignment 2

Submitted To:

Sir Waqas Saleem

Submitted By:

Hamail Fatima

Roll No:

2023-BSE-023

Section:

5(A)

Assignment # 02

Part 1: Infrastructure Setup (25 marks)

1.1 Project Structure (5 marks)

Create a well-organized Terraform project with the following structure:

Assignment2/

```
|—— main.tf
|—— variables.tf
|—— outputs.tf
|—— locals.tf
|—— terraform.tfvars
|—— .gitignore
|—— modules/
|   |—— networking/
|   |   |—— main.tf
|   |   |—— variables.tf
|   |   |—— outputs.tf
|   |—— security/
|   |   |—— main.tf
|   |   |—— variables.tf
|   |   |—— outputs.tf
|   |—— webserver/
|   |   |—— main.tf
|   |   |—— variables.tf
|   |   |—— outputs.tf
|—— scripts/
|   |—— nginx-setup.sh
|   |—— apache-setup.sh
|—— README.md
```

Tasks:

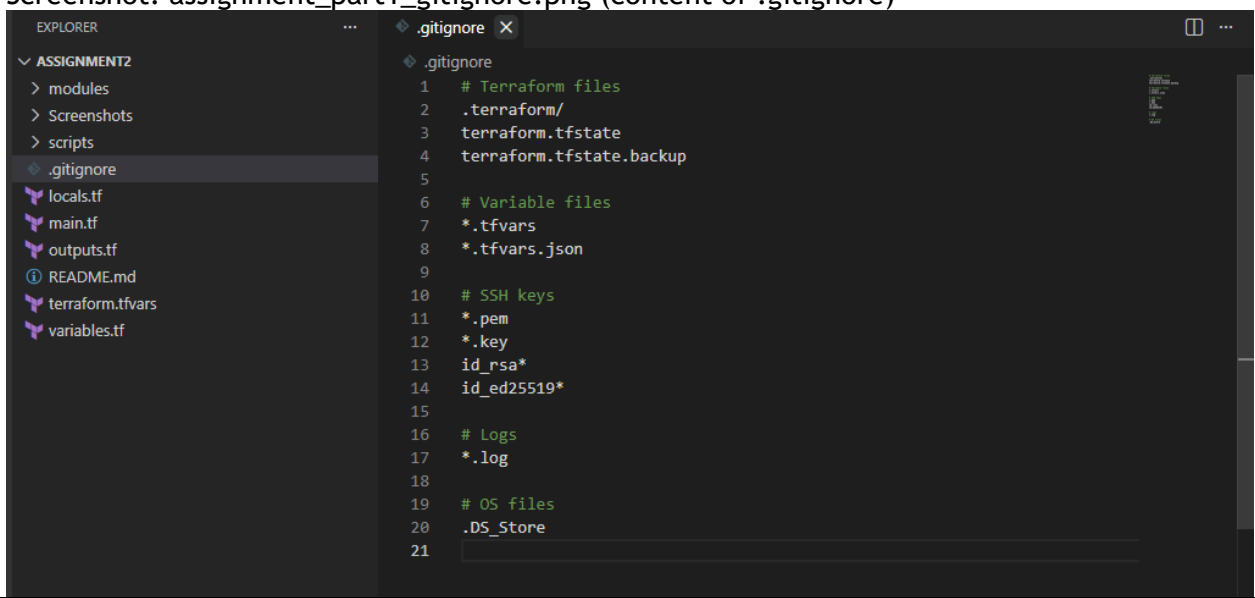
- Create all necessary files and directories
- Implement proper .gitignore to exclude sensitive files
- Document the project structure in README.md

Deliverables:

- Screenshot: assignment_part1_project_structure.png (tree command output)

```
D:.\n  .gitignore\n  locals.tf\n  main.tf\n  outputs.tf\n  README.md\n  terraform.tfvars\n  variables.tf\n\nmodules\n  networking\n    main.tf\n    outputs.tf\n    variables.tf\n  security\n    main.tf\n    outputs.tf\n    variables.tf\n  webserver\n    main.tf\n    outputs.tf\n    variables.tf\n\nscripts\n  apache-setup.sh\n  nginx-setup.sh\n\nPS D:\\Assignment2> |
```

- Screenshot: assignment_part1_gitignore.png (content of .gitignore)



1.2 Variable Configuration (5 marks)

Define all required variables in variables.tf:

Required Variables:

- vpc_cidr_block (string)
- subnet_cidr_block (string)
- availability_zone (string)
- env_prefix (string)
- instance_type (string)
- public_key (string)
- private_key (string)
- backend_servers (list of objects with name and script_path)

Tasks:

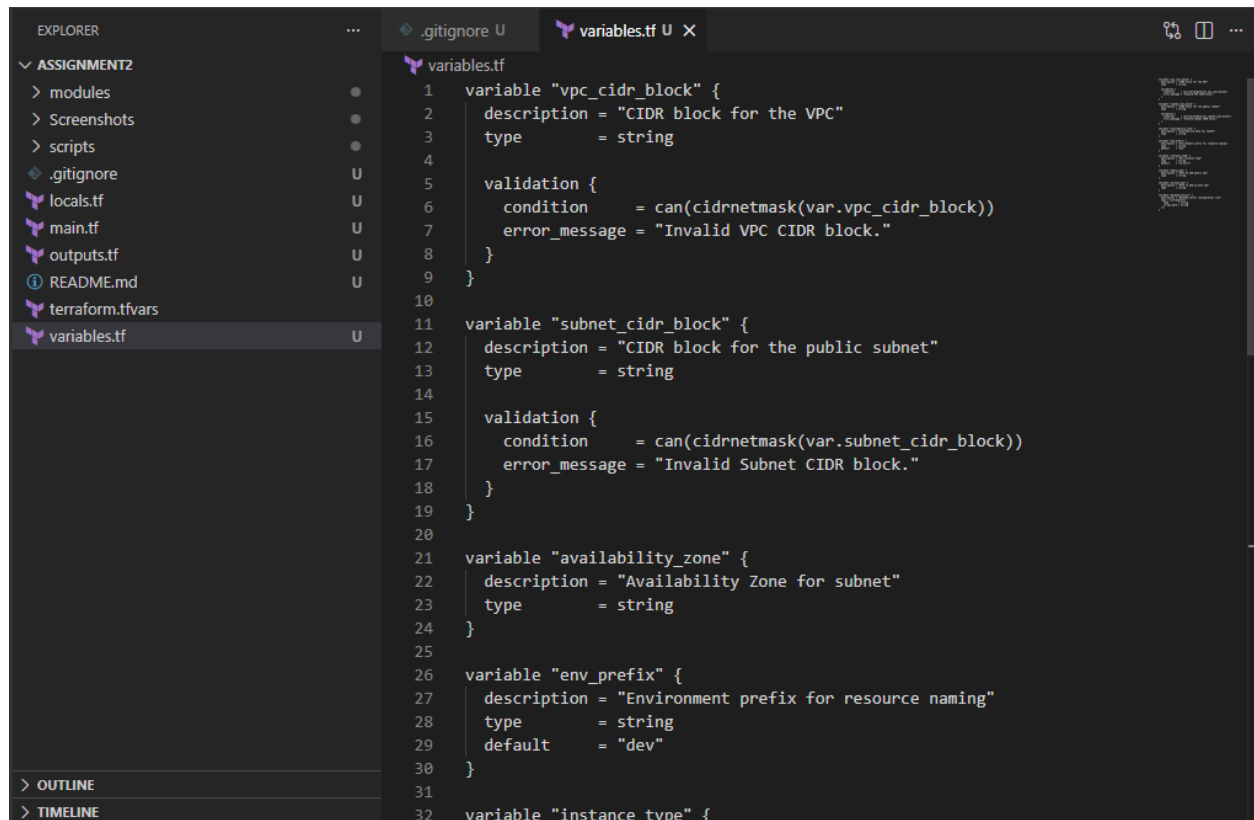
- Add validation rules for CIDR blocks
- Add descriptions for all variables
- Set appropriate defaults where applicable
- Populate terraform.tfvars with your values

Sample terraform.tfvars structure:

```
vpc_cidr_block    = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone = "me-central-1a"
env_prefix        = "prod"
instance_type     = "t3.micro"
public_key        = "~/.ssh/id_ed25519.pub"
private_key       = "~/.ssh/id_ed25519"
```

Deliverables:

- Screenshot: assignment_part1_variables_tf.png



```
1 variable "vpc_cidr_block" {
2   description = "CIDR block for the VPC"
3   type       = string
4
5   validation {
6     condition = can(cidrnetmask(var.vpc_cidr_block))
7     error_message = "Invalid VPC CIDR block."
8   }
9 }
10
11 variable "subnet_cidr_block" {
12   description = "CIDR block for the public subnet"
13   type       = string
14
15   validation {
16     condition = can(cidrnetmask(var.subnet_cidr_block))
17     error_message = "Invalid Subnet CIDR block."
18   }
19 }
20
21 variable "availability_zone" {
22   description = "Availability Zone for subnet"
23   type       = string
24 }
25
26 variable "env_prefix" {
27   description = "Environment prefix for resource naming"
28   type       = string
29   default    = "dev"
30 }
31
32 variable "instance_type" {
```

- Screenshot: assignment_part1_terraform_tfvars.png



1.3 Networking Module (5 marks)

Create a networking module that provisions:

- VPC with specified CIDR block
- Subnet with public IP assignment enabled
- Internet Gateway
- Route table with default route to IGW
- Associate route table with subnet

Module Location: modules/networking/

Required Outputs:

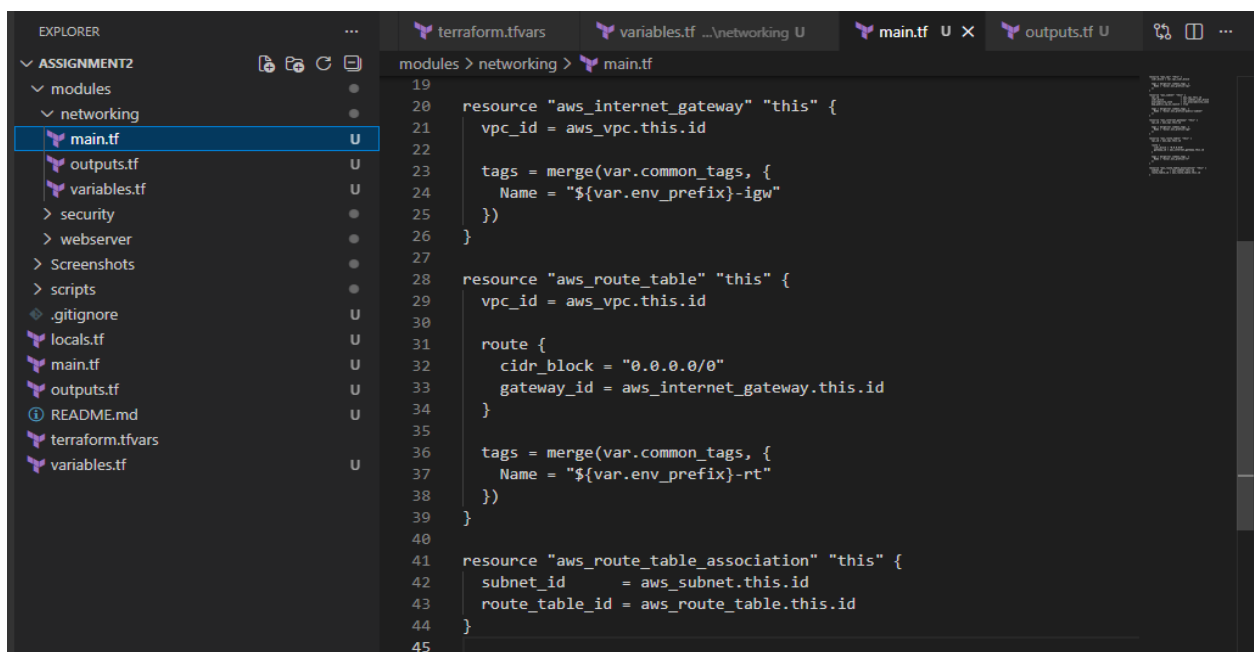
- vpc_id
- subnet_id
- igw_id
- route_table_id

Tasks:

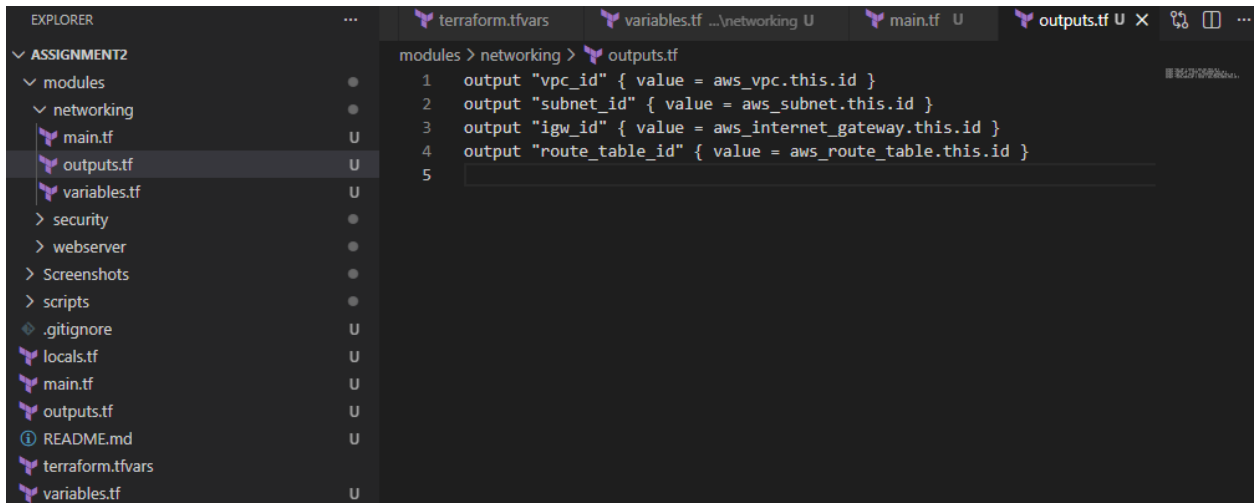
- Create VPC resource
- Create subnet with map_public_ip_on_launch = true
- Create and attach Internet Gateway
- Configure routing table
- Add proper tags to all resources using env_prefix

Deliverables:

- Screenshot: assignment_part1_networking_module_main.png



- Screenshot: assignment_part1_networking_module_outputs.png



1.4 Security Module (5 marks)

Create a security module that provisions:

Two Security Groups:

1. Nginx Security Group (for reverse proxy/load balancer):

- Ingress: Port 22 (SSH) from your IP only
- Ingress: Port 80 (HTTP) from anywhere (0.0.0.0/0)
- Ingress: Port 443 (HTTPS) from anywhere (0.0.0.0/0)
- Egress: All traffic

2. Backend Security Group (for web servers):

- Ingress: Port 22 (SSH) from your IP only
- Ingress: Port 80 (HTTP) from Nginx security group only
- Egress: All traffic

Module Location: modules/security/

Required Variables:

- vpc_id
- env_prefix
- my_ip

Required Outputs:

- nginx_sg_id
- backend_sg_id

Tasks:

- Create Nginx security group with appropriate rules
- Create backend security group with appropriate rules
- Use security group IDs for backend ingress (not CIDR blocks)
- Add descriptive names and tags

Deliverables:

- Screenshot: assignment_part1_security_module. Png

```

1 resource "aws_security_group" "nginx_sg" {
2   name = "${var.env_prefix}-nginx-sg"
3   vpc_id = var.vpc_id
4
5   ingress {
6     from_port = 22
7     to_port = 22
8     protocol = "tcp"
9     cidr_blocks = [var.my_ip]
10  }
11
12  ingress {
13    from_port = 80
14    to_port = 80
15    protocol = "tcp"
16    cidr_blocks = ["0.0.0.0/0"]
17  }
18
19  ingress {
20    from_port = 443
21    to_port = 443
22    protocol = "tcp"
23    cidr_blocks = ["0.0.0.0/0"]
24  }
25
26  egress {
27    from_port = 0
28    to_port = 0
29    protocol = "-1"
30    cidr_blocks = ["0.0.0.0/0"]
31  }
32

```

- Screenshot: assignment_part1_security_groups_console.png (AWS Console)

Name	Security group ID	Security group name	VPC ID	Description
prod-backend-sg	sg-0d93a4d788cd2f83c	prod-backend-sg	vpc-02e5f1f03e6b88a38	Security group for prod-backend-sg
-	sg-0c69b84157c331a33	default	vpc-02e5f1f03e6b88a38	default VPC security group
prod-nginx-sg	sg-08070cdd21347de2	prod-nginx-sg	vpc-02e5f1f03e6b88a38	Security group for prod-nginx-sg
-	sg-0679153ce974003ce	default	vpc-0a2cc4bba16a7c9b	default VPC security group

1.5 Locals Configuration (5 marks)

Create locals.tf with:

- Dynamic IP detection for my_ip
- Resource naming conventions
- Common tags
- Backend server configurations

Example:

```

locals {
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"

  common_tags = {
    Environment = var.env_prefix
    Project     = "Assignment-2"
    ManagedBy   = "Terraform"
  }
}

```

```

backend_servers = [
  {
    name      = "web-1"
    suffix    = "1"
    script_path = "./scripts/apache-setup.sh"
  },
  {
    name      = "web-2"
    suffix    = "2"
    script_path = "./scripts/apache-setup.sh"
  },
  {
    name      = "web-3"
    suffix    = "3"
    script_path = "./scripts/apache-setup.sh"
  }
]

```

```

data "http" "my_ip" {
  url = "https://icanhazip.com"
}

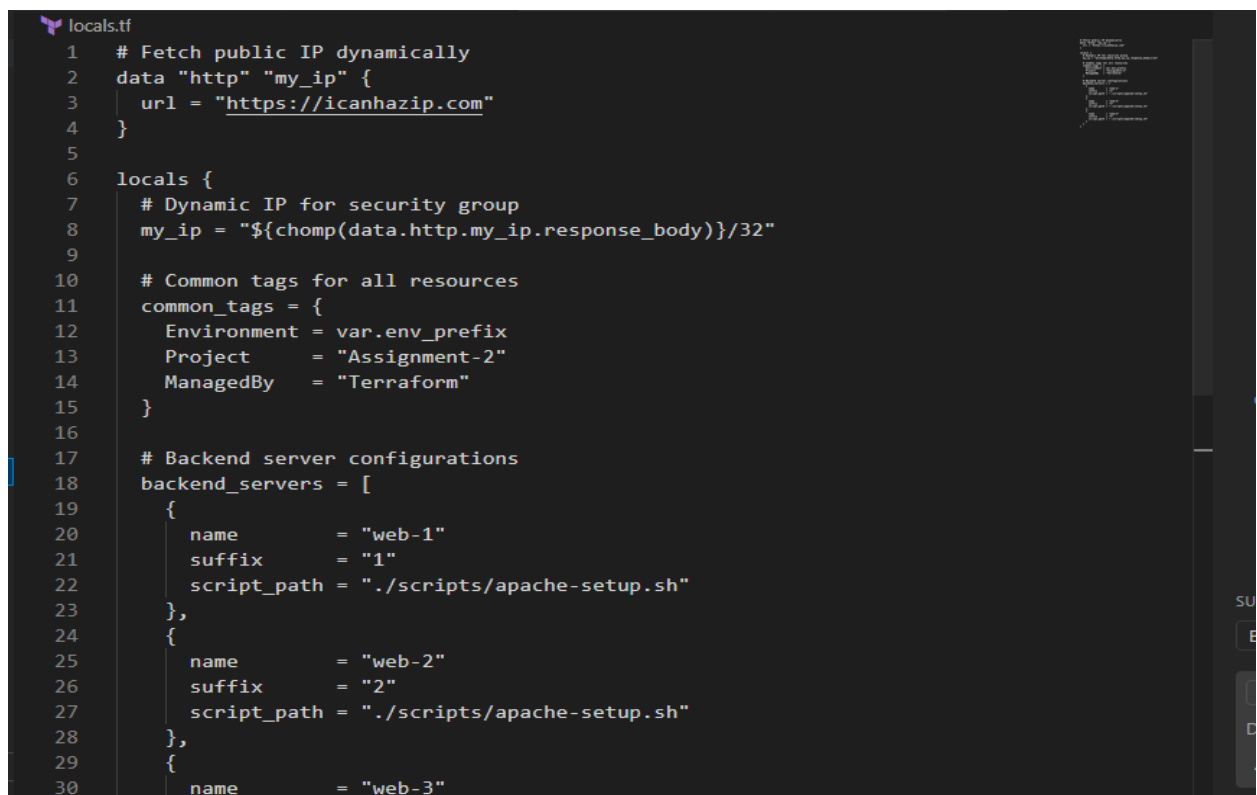
```

Tasks:

- Implement dynamic IP detection
- Define common tags
- Define backend server list
- Add any other reusable local values

Deliverables:

- Screenshot: assignment_part1_locals_tf.png



```

locals.tf
1  # Fetch public IP dynamically
2  data "http" "my_ip" {
3    url = "https://icanhazip.com"
4  }
5
6  locals {
7    # Dynamic IP for security group
8    my_ip = "${chomp(data.http.my_ip.response_body)}/32"
9
10   # Common tags for all resources
11   common_tags = {
12     Environment = var.env_prefix
13     Project     = "Assignment-2"
14     ManagedBy   = "Terraform"
15   }
16
17   # Backend server configurations
18   backend_servers = [
19     {
20       name      = "web-1"
21       suffix    = "1"
22       script_path = "./scripts/apache-setup.sh"
23     },
24     {
25       name      = "web-2"
26       suffix    = "2"
27       script_path = "./scripts/apache-setup.sh"
28     },
29     {
30       name      = "web-3"

```

Part 2: Webserver Module (15 marks)

2.1 Module Design (10 marks)

Create a reusable webserver module in modules/webserver/

Module Requirements:

Variables:

- env_prefix
- instance_name
- instance_type
- availability_zone
- vpc_id
- subnet_id
- security_group_id
- public_key
- script_path
- instance_suffix
- common_tags

Resources to Create:

1. AWS Key Pair (unique per instance using suffix)
2. EC2 Instance with:
 - Specified AMI (Amazon Linux 2023)
 - Public IP enabled
 - User data from script file
 - Proper tags

Outputs:

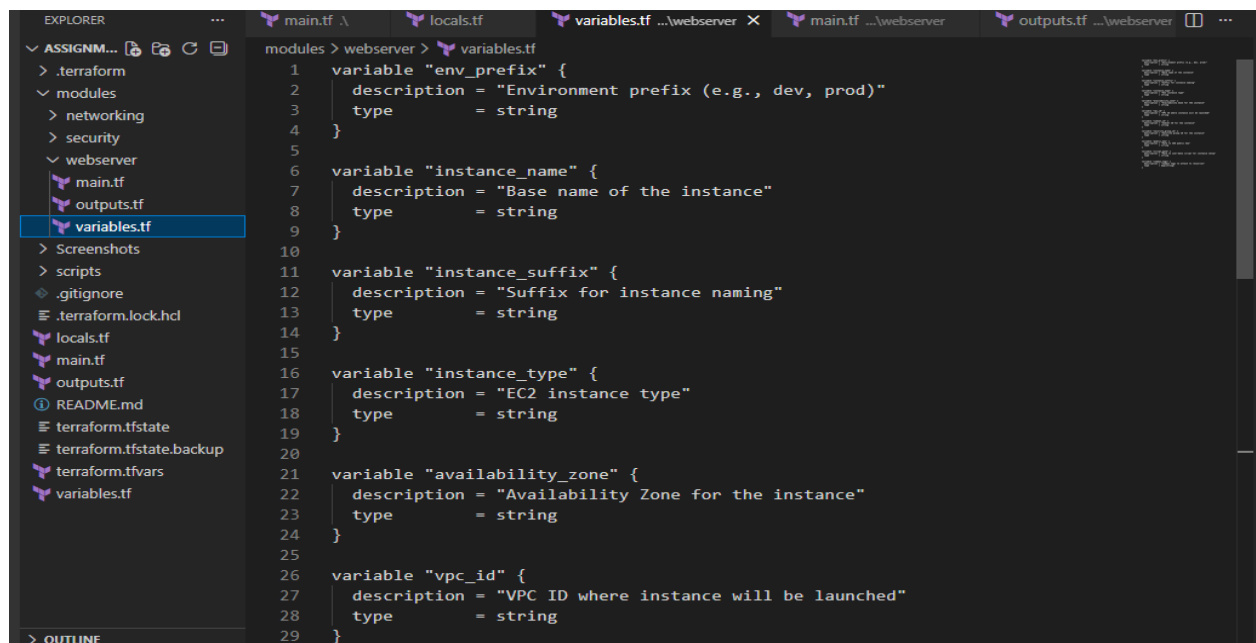
- instance_id
- public_ip
- private_ip

Tasks:

- Create variables.tf with all required variables
- Create main.tf with key pair and instance resources
- Create outputs.tf with all required outputs
- Ensure module is reusable for both Nginx and backend servers

Deliverables:

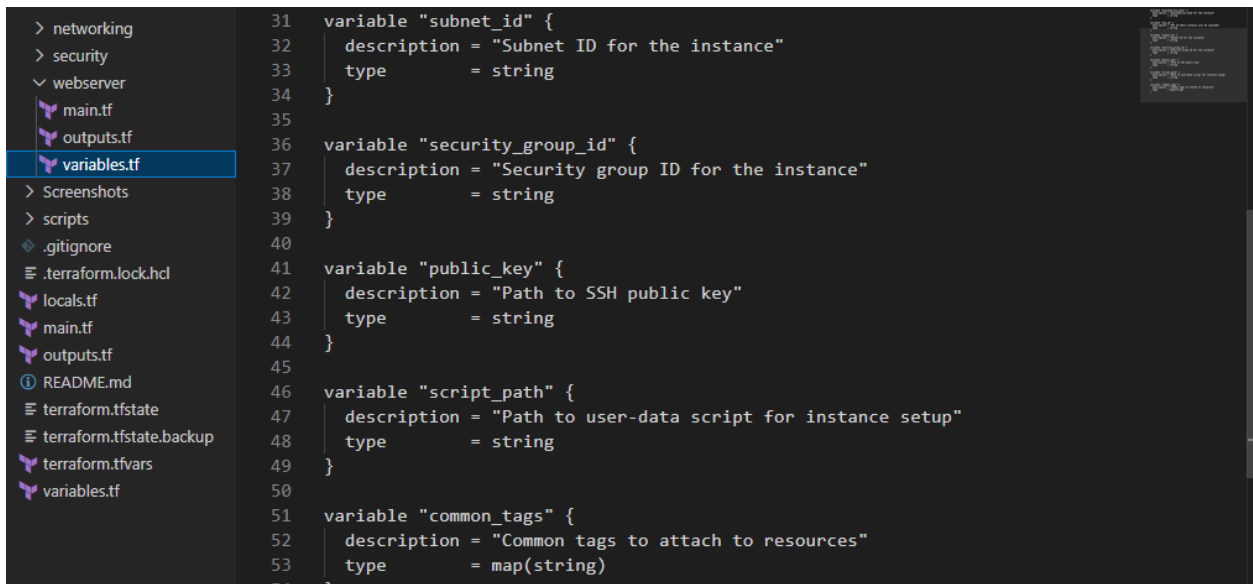
- Screenshot: assignment_part2_webserver_module_variables.png



The screenshot shows a code editor with the following structure:

- EXPLORER:** A file tree on the left showing the project structure. The 'modules' directory is expanded, and 'webserver' is selected. Inside 'webserver', 'variables.tf' is highlighted.
- main.tf:** The active file in the editor, showing the following Terraform code:

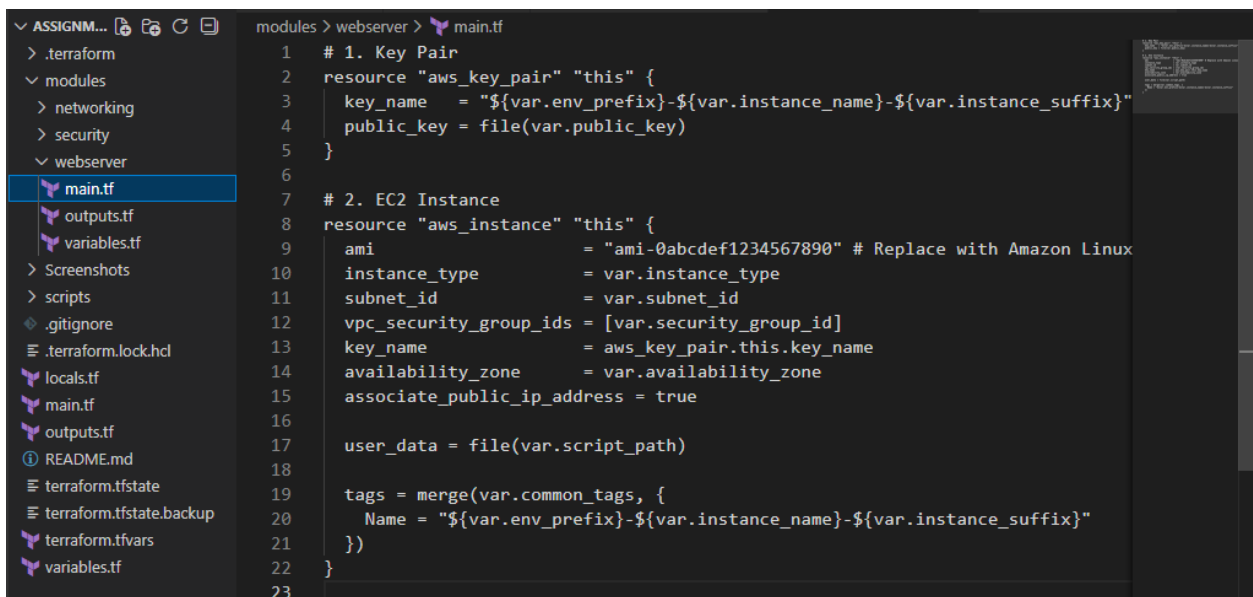
```
1 variable "env_prefix" {  
2   description = "Environment prefix (e.g., dev, prod)"  
3   type       = string  
4 }  
5  
6 variable "instance_name" {  
7   description = "Base name of the instance"  
8   type       = string  
9 }  
10  
11 variable "instance_suffix" {  
12   description = "Suffix for instance naming"  
13   type       = string  
14 }  
15  
16 variable "instance_type" {  
17   description = "EC2 instance type"  
18   type       = string  
19 }  
20  
21 variable "availability_zone" {  
22   description = "Availability Zone for the instance"  
23   type       = string  
24 }  
25  
26 variable "vpc_id" {  
27   description = "VPC ID where instance will be launched"  
28   type       = string  
29 }
```



This screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'networking', 'security', and 'webserver'. The 'webserver' folder is expanded, showing files like 'main.tf', 'outputs.tf', and 'variables.tf'. The 'variables.tf' file is selected and its content is displayed in the code editor. The code defines several variables for an AWS instance setup.

```
31 variable "subnet_id" {
32   description = "Subnet ID for the instance"
33   type        = string
34 }
35
36 variable "security_group_id" {
37   description = "Security group ID for the instance"
38   type        = string
39 }
40
41 variable "public_key" {
42   description = "Path to SSH public key"
43   type        = string
44 }
45
46 variable "script_path" {
47   description = "Path to user-data script for instance setup"
48   type        = string
49 }
50
51 variable "common_tags" {
52   description = "Common tags to attach to resources"
53   type        = map(string)
```

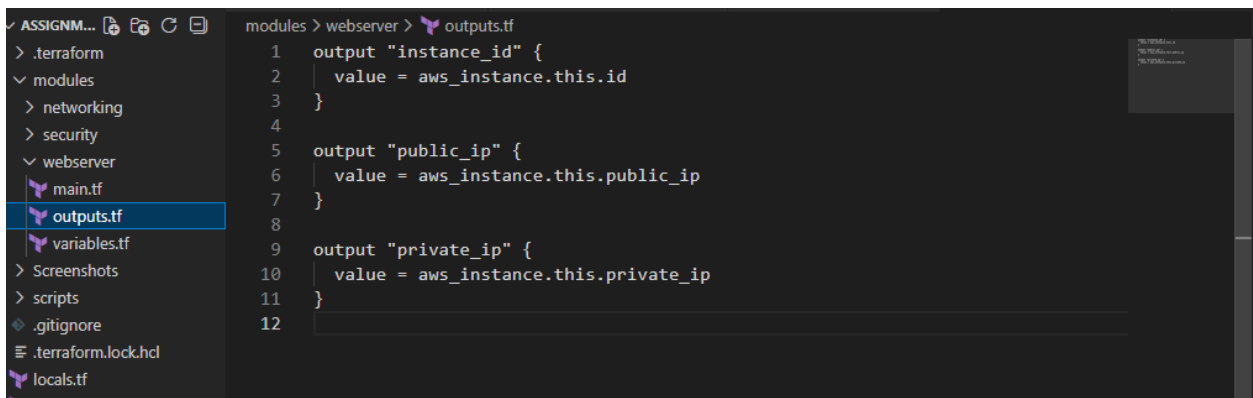
- Screenshot: assignment_part2_webserver_module_main.png



This screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'terraform', 'modules', and 'webserver'. The 'webserver' folder is expanded, showing files like 'main.tf', 'outputs.tf', and 'variables.tf'. The 'main.tf' file is selected and its content is displayed in the code editor. The code defines two resources: an AWS Key Pair and an AWS EC2 Instance.

```
modules > webserver > main.tf
1  # 1. Key Pair
2  resource "aws_key_pair" "this" {
3    key_name   = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"
4    public_key = file(var.public_key)
5  }
6
7  # 2. EC2 Instance
8  resource "aws_instance" "this" {
9    ami              = "ami-0abcdef1234567890" # Replace with Amazon Linux
10   instance_type    = var.instance_type
11   subnet_id        = var.subnet_id
12   vpc_security_group_ids = [var.security_group_id]
13   key_name          = aws_key_pair.this.key_name
14   availability_zone = var.availability_zone
15   associate_public_ip_address = true
16
17   user_data = file(var.script_path)
18
19   tags = merge(var.common_tags, {
20     Name = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"
21   })
22 }
23
```

- Screenshot: assignment_part2_webserver_module_outputs.png



This screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'terraform', 'modules', and 'webserver'. The 'webserver' folder is expanded, showing files like 'main.tf', 'outputs.tf', and 'variables.tf'. The 'outputs.tf' file is selected and its content is displayed in the code editor. The code defines three outputs: instance_id, public_ip, and private_ip.

```
modules > webserver > outputs.tf
1  output "instance_id" {
2    value = aws_instance.this.id
3  }
4
5  output "public_ip" {
6    value = aws_instance.this.public_ip
7  }
8
9  output "private_ip" {
10   value = aws_instance.this.private_ip
11 }
12
```

2.2 Module Usage (5 marks)

In root main.tf, instantiate the webserver module for:

1. One Nginx server (using nginx-setup.sh)
2. Three backend servers (web-1, web-2, web-3 using apache-setup.sh)

Use dynamic blocks or for_each for backend servers:

Nginx Server

```
module "nginx_server" {
  source          = "./modules/webserver"
  env_prefix      = var.env_prefix
  instance_name   = "nginx-proxy"
  instance_type   = var.instance_type
  availability_zone = var.availability_zone
  vpc_id          = module.networking.vpc_id
  subnet_id       = module.networking.subnet_id
  security_group_id = module.security.nginx_sg_id
  public_key      = var.public_key
  script_path     = "./scripts/nginx-setup.sh"
  instance_suffix = "nginx"
  common_tags     = local.common_tags
}
```

Backend Servers (use for_each or count)

```
module "backend_servers" {
  for_each = { for idx, server in local.backend_servers : server.name => server }

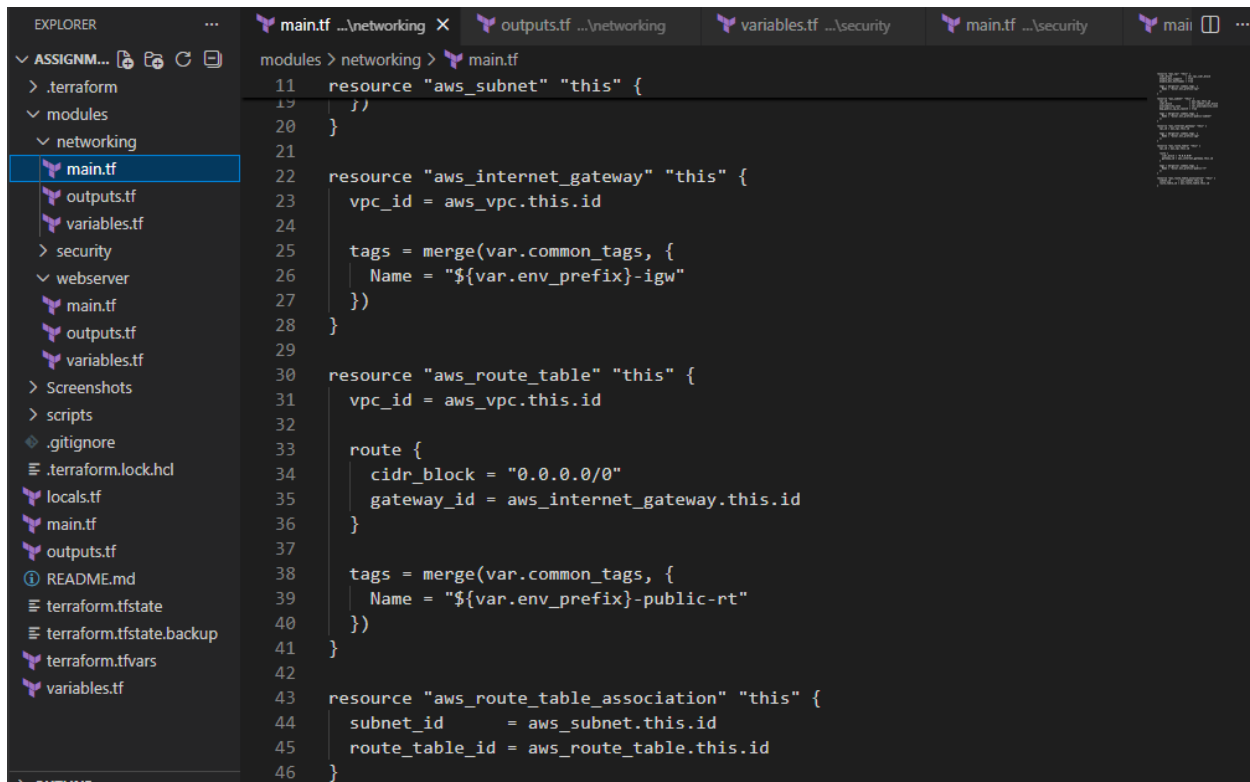
  source          = "./modules/webserver"
  env_prefix      = var.env_prefix
  instance_name   = each.value.name
  instance_type   = var.instance_type
  availability_zone = var.availability_zone
  vpc_id          = module.networking.vpc_id
  subnet_id       = module.networking.subnet_id
  security_group_id = module.security.backend_sg_id
  public_key      = var.public_key
  script_path     = each.value.script_path
  instance_suffix = each.value.suffix
  common_tags     = local.common_tags
}
```

Tasks:

- Create Nginx server module instance
- Create backend server module instances using for_each
- Ensure proper security group assignment
- Pass all required variables

Deliverables:

- Screenshot: assignment_part2_main_tf_modules.png



Part 3: Server Configuration Scripts (20 marks)

3.1 Apache Backend Server Script (10 marks)

Create scripts/apache-setup.sh that:

- Updates system packages
- Installs Apache HTTP Server
- Starts and enables Apache service
- Creates custom HTML page displaying:
 - Server name/hostname
 - Private IP address
 - Public IP address
 - Public DNS hostname
 - Custom message indicating which backend server (web-1, web-2, or web-3)
 - Timestamp of deployment
 - Server status (Primary/Backup)

Enhanced Script Requirements:

```
#!/bin/bash
```

```
set -e
```

```
# Update system
```

```
yum update -y
```

```
# Install Apache
```

```
yum install httpd -y
```

```
# Start and enable Apache
```

```
systemctl start httpd
```

```
systemctl enable httpd
```

```
# Get metadata token (IMDSv2)
```

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
```

```
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
```

```

# Get instance metadata
PRIVATE_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/local-ipv4)
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/public-ipv4)
PUBLIC_DNS=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/public-hostname)
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)

# Set hostname
hostnamectl set-hostname myapp-webserver

# Create custom HTML page
cat > /var/www/html/index.html <<EOF
<!DOCTYPE html>
<html>
<head>
  <title>Backend Web Server</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white;
    }
    .container {
      background: rgba(255, 255, 255, 0.1);
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.37);
    }
    h1 { color: #fff; text-shadow: 2px 2px 4px rgba(0,0,0,0.3); }
    .info { margin: 15px 0; padding: 10px; background: rgba(255,255,255,0.2); border-
radius: 5px; }
    .label { font-weight: bold; color: #ffd700; }
  </style>
</head>
<body>
  <div class="container">
    <h1> 🚀 Backend Web Server - Assignment 2</h1>
    <div class="info"><span class="label">Hostname:</span> $(hostname)</div>
    <div class="info"><span class="label">Instance ID:</span> $INSTANCE_ID</div>
    <div class="info"><span class="label">Private IP:</span> $PRIVATE_IP</div>
    <div class="info"><span class="label">Public IP:</span> $PUBLIC_IP</div>
    <div class="info"><span class="label">Public DNS:</span> $PUBLIC_DNS</div>
    <div class="info"><span class="label">Deployed: </span> $(date)</div>
    <div class="info"><span class="label">Status:</span> ✅ Active and Running</div>
    <div class="info"><span class="label">Managed By:</span> Terraform</div>
  </div>
</body>
</html>
EOF

```

Set permissions

chmod 644 /var/www/html/index.html

echo "Apache setup completed successfully!"

Tasks:

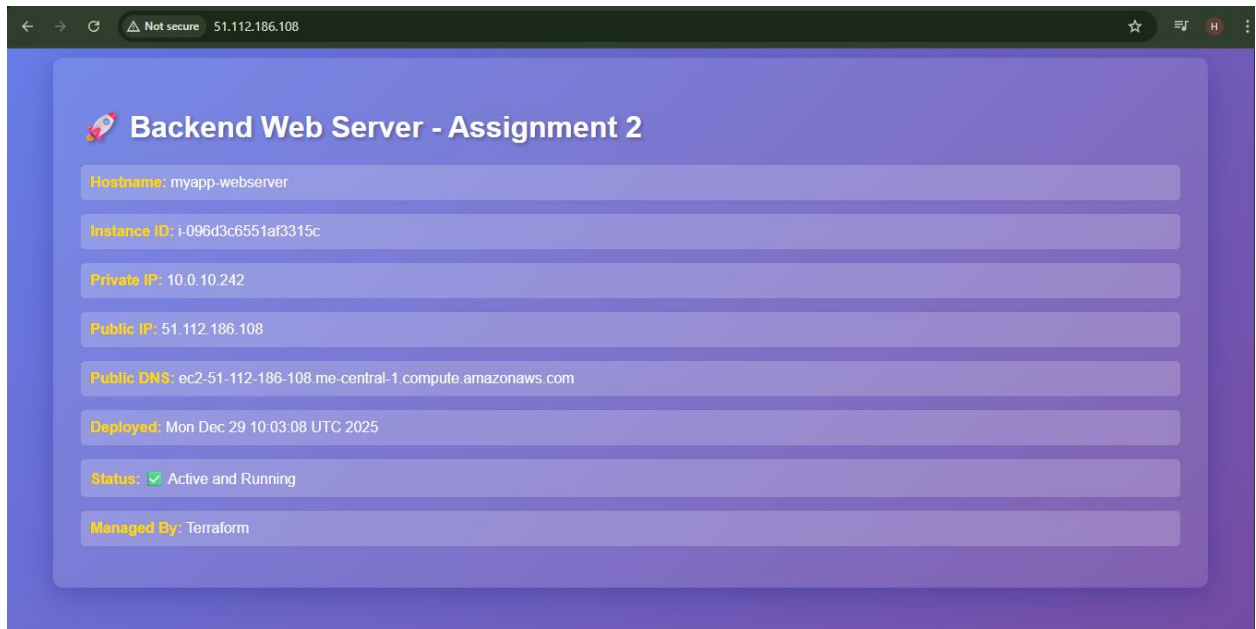
- Create the script with all required features
- Ensure it uses IMDSv2 for metadata
- Create visually appealing HTML output
- Make script executable
- Test script independently

Deliverables:

- Screenshot: assignment_part3_apache_script.png

```
scripts > $ apache-setup.sh
1  #!/bin/bash
2  set -e
3
4  # -----
5  # Apache Backend Setup Script
6  # -----
7
8  # Update system packages
9  yum update -y
10
11 # Install Apache HTTP Server
12 yum install httpd -y
13
14 # Start and enable Apache
15 systemctl enable --now httpd
16
17 # -----
18 # Fetch instance metadata (IMDSv2)
19 # -----
20 TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
21 -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
22
23 PRIVATE_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
24 http://169.254.169.254/latest/meta-data/local-ipv4)
25 PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
26 http://169.254.169.254/latest/meta-data/public-ipv4)
27 PUBLIC_DNS=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
28 http://169.254.169.254/latest/meta-data/public-hostname)
29 INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
30 http://169.254.169.254/latest/meta-data/instance-id)
31
32 # -----
33 # Set hostname
34 # -----
35 hostnamectl set-hostname myapp-webserver
36
37 # -----
38 # Create custom HTML page
39 # -----
40 cat > /var/www/html/index.html <<EOF
41 <!DOCTYPE html>
42 <html>
43 <head>
44   <title>Backend Web Server</title>
45   <style>
46     body {
47       font-family: Arial, sans-serif;
48       margin: 50px;
49       background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
50       color: white;
51     }
52     .container {
53       background: rgba(255, 255, 255, 0.1);
54       padding: 30px;
55       border-radius: 10px;
56       box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.37);
57     }
58     h1 { color: #ffff; text-shadow: 2px 2px 4px rgba(0,0,0,0.3); }
59     .info { margin: 15px 0; padding: 10px; background: rgba(255,255,255,0.2); border-radi
60     .label { font-weight: bold; color: #ffd700; }
61   </style>
62 </html>
63 EOF
```

- Screenshot: assignment_part3_backend_webpage.png (browser showing backend server page)



3.2 Nginx Server Setup Script (10 marks)

Create scripts/nginx-setup.sh that:

- Updates system packages
- Installs Nginx
- Generates self-signed SSL certificate
- Configures Nginx with:
 - HTTPS on port 443
 - HTTP to HTTPS redirect
 - Upstream backend servers (web-1, web-2, web-3)
 - Load balancing configuration
 - Caching configuration
 - Proper logging
 - Security headers

Enhanced Script Template:

```
#!/bin/bash
```

```
set -e
```

```
# Update and install Nginx
```

```
yum update -y
```

```
yum install -y nginx openssl
```

```
systemctl start nginx
```

```
systemctl enable nginx
```

```
# Create SSL directories
```

```
mkdir -p /etc/ssl/private
```

```
mkdir -p /etc/ssl/certs
```

```
# Get metadata token
```

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
```

```
# Get public IP
```

```
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4)
```

```
# Generate self-signed certificate
```

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/CN=$PUBLIC_IP" \
-addext "subjectAltName=IP:$PUBLIC_IP" \
-addext "basicConstraints=CA:FALSE" \
-addext "keyUsage=digitalSignature,keyEncipherment" \
-addext "extendedKeyUsage=serverAuth"
```

```
echo "Self-signed certificate created for IP: $PUBLIC_IP"
```

```
# Backup original config
```

```
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
```

```
# Create Nginx configuration
```

```
# Note: Backend IPs will be added manually after deployment
```

```
cat > /etc/nginx/nginx.conf <<'EOF'
```

```
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;
```

```
events {
    worker_connections 1024;
}
```

```
http {
    # Logging
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for" '
        'Cache: $upstream_cache_status';
```

```
access_log /var/log/nginx/access.log main;
```

```
# Basic settings
```

```
sendfile on;
tcp_nopush on;
keepalive_timeout 65;
types_hash_max_size 4096;
```

```
include /etc/nginx/mime.types;
default_type application/octet-stream;
```

```
# Gzip compression
```

```
gzip on;
gzip_vary on;
gzip_types text/plain text/css application/json application/javascript text/xml
application/xml;
```

```
# Cache configuration
```

```

proxy_cache_path /var/cache/nginx
    levels=1:2
    keys_zone=my_cache:10m
    max_size=1g
    inactive=60m
    use_temp_path=off;

# Upstream backend servers
# PLACEHOLDER: Update these IPs after deployment
upstream backend_servers {
    # Primary servers (active load balancing)
    server BACKEND_IP_1:80;
    server BACKEND_IP_2:80;

    # Backup server (only used when primary servers are down)
    server BACKEND_IP_3:80 backup;
}

# HTTPS Server
server {
    listen 443 ssl http2;
    server_name _;

    # SSL Configuration
    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    # Security Headers
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    # Proxy settings
    location / {
        proxy_pass http://backend_servers;

        # Proxy headers
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Cache settings
        proxy_cache my_cache;
        proxy_cache_valid 200 60m;
        proxy_cache_valid 404 10m;
        proxy_cache_key "$scheme$request_method$host$request_uri";
        proxy_cache_bypass $http_cache_control;
        add_header X-Cache-Status $upstream_cache_status;
    }
}

```

```

        # Timeouts
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # Health check endpoint
    location /health {
        access_log off;
        return 200 "Nginx is healthy\n";
        add_header Content-Type text/plain;
    }
}

# HTTP Server (redirect to HTTPS)
server {
    listen 80;
    server_name _;

    location / {
        return 301 https://$host$request_uri;
    }

    # Allow health checks over HTTP
    location /health {
        access_log off;
        return 200 "Nginx is healthy\n";
        add_header Content-Type text/plain;
    }
}
}
EOF

# Create cache directory
mkdir -p /var/cache/nginx
chown -R nginx:nginx /var/cache/nginx

# Test and restart Nginx
nginx -t && systemctl restart nginx

echo "Nginx setup completed successfully!"
echo "Remember to update backend server IPs in /etc/nginx/nginx.conf"

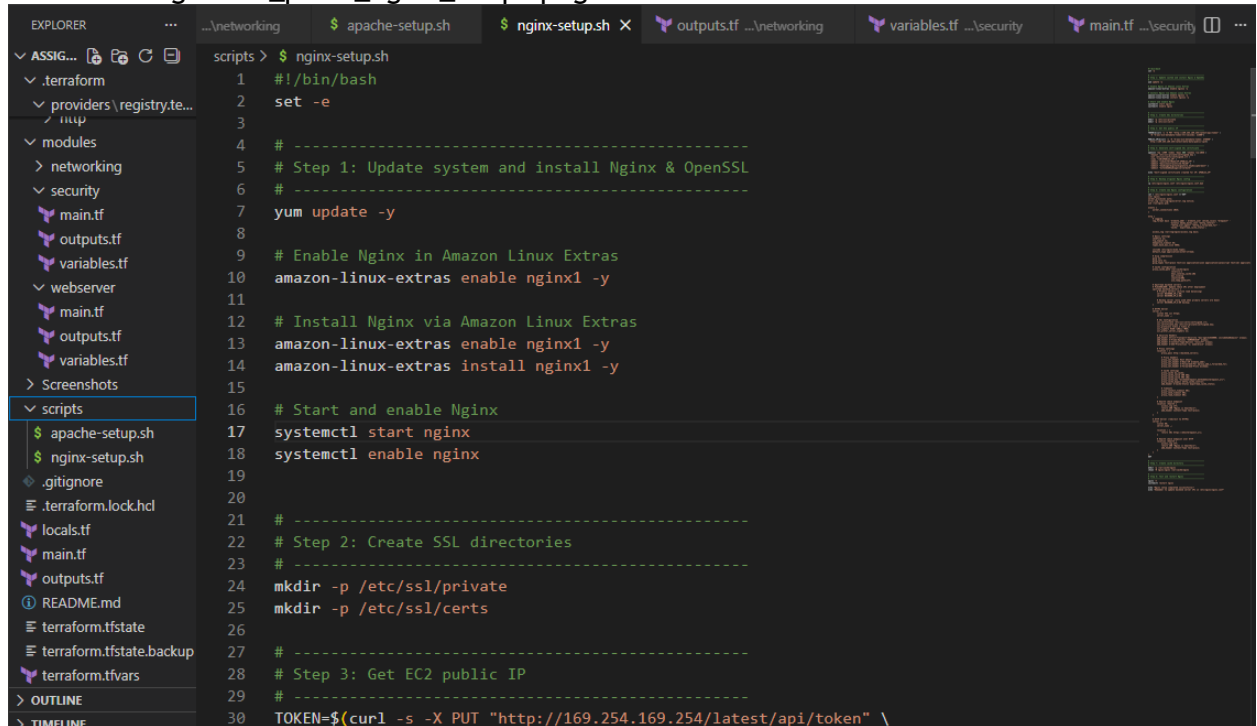
```

Tasks:

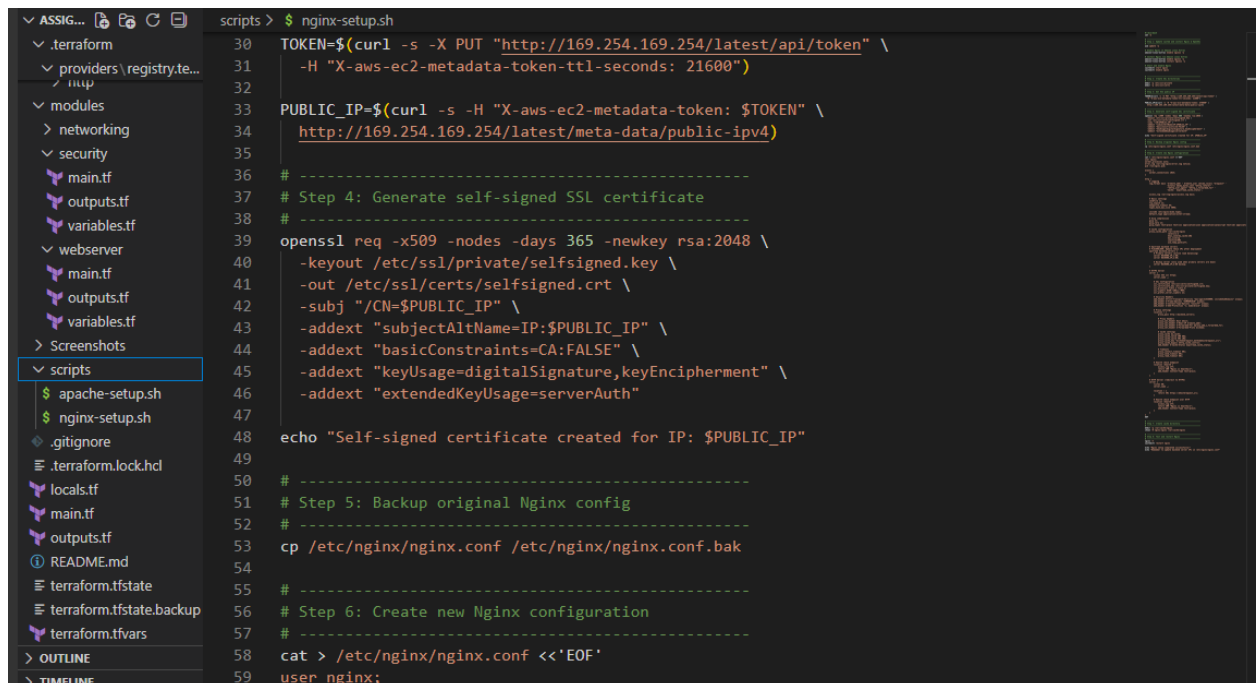
- Create script with SSL certificate generation
- Configure upstream with placeholder IPs
- Implement caching
- Add security headers
- Configure HTTP to HTTPS redirect
- Add health check endpoint

Deliverables:

- Screenshot: assignment_part3_nginx_script.png



```
EXPLORER  ...  ...networking  $ apache-setup.sh  $ nginx-setup.sh  X  outputs.tf ...networking  variables.tf ...security  main.tf ...security  ...  
ASSIG...  ...  ...  ...  
  .terraform  ...  
    providers\registry.te...  
      .tfvars  
  modules  
    > networking  
    > security  
      main.tf  
      outputs.tf  
      variables.tf  
  > webserver  
    main.tf  
    outputs.tf  
    variables.tf  
  > Screenshots  
  > scripts  
    $ apache-setup.sh  
    $ nginx-setup.sh  
    .gitignore  
    .terraform.lock.hcl  
    locals.tf  
    main.tf  
    outputs.tf  
    README.md  
    terraform.tfstate  
    terraform.tfstate.backup  
    terraform.tfvars  
  > OUTLINE  
  > TIMELINE  
scripts > $ nginx-setup.sh  
1  #!/bin/bash  
2  set -e  
3  
4  # -----  
5  # Step 1: Update system and install Nginx & OpenSSL  
6  # -----  
7  yum update -y  
8  
9  # Enable Nginx in Amazon Linux Extras  
10 amazon-linux-extras enable nginx1 -y  
11  
12 # Install Nginx via Amazon Linux Extras  
13 amazon-linux-extras enable nginx1 -y  
14 amazon-linux-extras install nginx1 -y  
15  
16 # Start and enable Nginx  
17 systemctl start nginx  
18 systemctl enable nginx  
19  
20  
21 # -----  
22 # Step 2: Create SSL directories  
23 # -----  
24 mkdir -p /etc/ssl/private  
25 mkdir -p /etc/ssl/certs  
26  
27 # -----  
28 # Step 3: Get EC2 public IP  
29 # -----  
30 TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
```



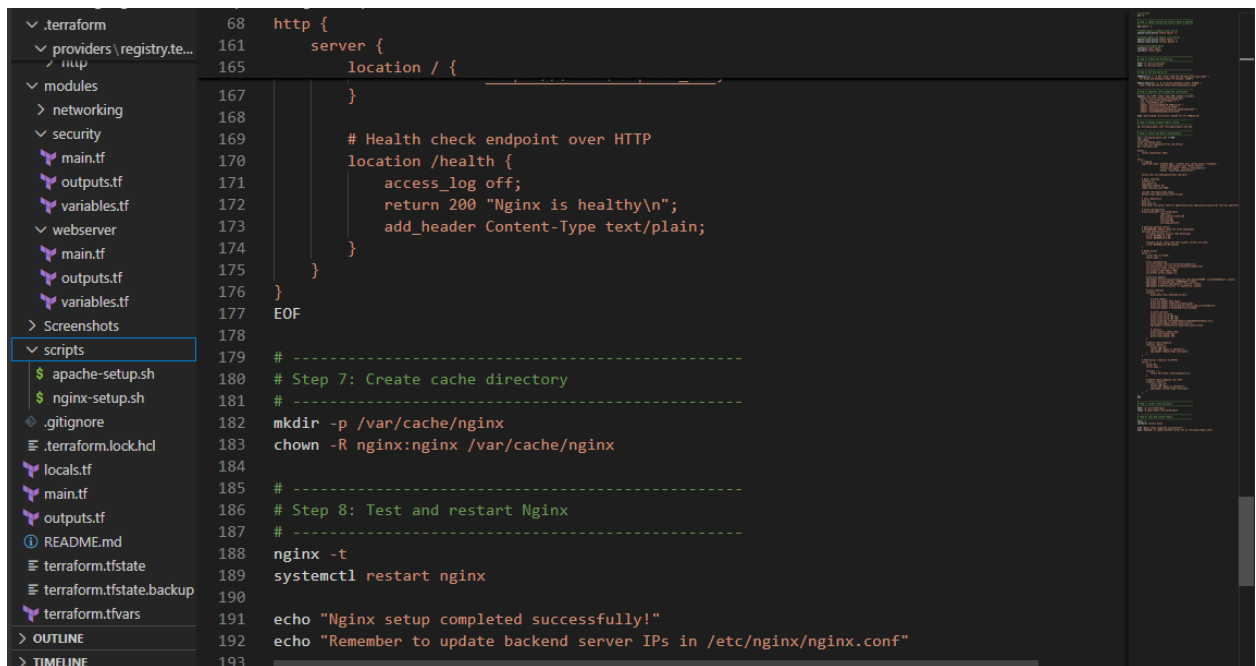
```
  > ASSIG...  ...  ...  ...  
  > .terraform  ...  
    providers\registry.te...  
      .tfvars  
  > modules  
    > networking  
    > security  
      main.tf  
      outputs.tf  
      variables.tf  
  > webserver  
    main.tf  
    outputs.tf  
    variables.tf  
  > Screenshots  
  > scripts  
    $ apache-setup.sh  
    $ nginx-setup.sh  
    .gitignore  
    .terraform.lock.hcl  
    locals.tf  
    main.tf  
    outputs.tf  
    README.md  
    terraform.tfstate  
    terraform.tfstate.backup  
    terraform.tfvars  
  > OUTLINE  
  > TIMELINE  
scripts > $ nginx-setup.sh  
30 TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \\  
31 -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")\  
32  
33 PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \  
34 http://169.254.169.254/latest/meta-data/public-ipv4)  
35  
36 # -----  
37 # Step 4: Generate self-signed SSL certificate  
38 # -----  
39 openssl req -x509 -nodes -days 365 -newkey rsa:2048 \  
40 -keyout /etc/ssl/private/selfsigned.key \  
41 -out /etc/ssl/certs/selfsigned.crt \  
42 -subj "/CN=$PUBLIC_IP" \  
43 -addext "subjectAltName=IP:$PUBLIC_IP" \  
44 -addext "basicConstraints=CA:FALSE" \  
45 -addext "keyUsage=digitalSignature,keyEncipherment" \  
46 -addext "extendedKeyUsage=serverAuth"  
47  
48 echo "Self-signed certificate created for IP: $PUBLIC_IP"  
49  
50 # -----  
51 # Step 5: Backup original Nginx config  
52 # -----  
53 cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak  
54  
55 # -----  
56 # Step 6: Create new Nginx configuration  
57 # -----  
58 cat > /etc/nginx/nginx.conf <<'EOF'  
59 user nginx;
```

```
scripts > $ nginx-setup.sh
59 user nginx;
60 worker_processes auto;
61 error_log /var/log/nginx/error.log notice;
62 pid /run/nginx.pid;
63
64 events {
65     worker_connections 1024;
66 }
67
68 http {
69     # Logging
70     log_format main '$remote_addr - $remote_user [$time_local] "$request" '
71                     '$status $body_bytes_sent "$http_referer" '
72                     '"$http_user_agent" "$http_x_forwarded_for" '
73                     'Cache: $upstream_cache_status';
74
75     access_log /var/log/nginx/access.log main;
76
77     # Basic settings
78     sendfile on;
79     tcp_nopush on;
80     keepalive_timeout 65;
81     types_hash_max_size 4096;
82
83     include /etc/nginx/mime.types;
84     default_type application/octet-stream;
85
86     # Gzip compression
87     gzip on;
88     gzip_vary on;
```

```
scripts > $ nginx-setup.sh
68 http {
88     gzip_vary on;
89     gzip_types text/plain text/css application/json application/javascript text/xml applicati
90
91     # Cache configuration
92     proxy_cache_path /var/cache/nginx
93         levels=1:2
94         keys_zone=my_cache:10m
95         max_size=1g
96         inactive=60m
97         use_temp_path=off;
98
99     # Upstream backend servers
100     # PLACEHOLDER: Update these IPs after deployment
101     upstream backend_servers {
102         # Primary servers (active load balancing)
103         server BACKEND_IP_1:80;
104         server BACKEND_IP_2:80;
105
106         # Backup server (only used when primary servers are down)
107         server BACKEND_IP_3:80 backup;
108     }
109
110     # HTTPS Server
111     server {
112         listen 443 ssl http2;
113         server_name _;
114
115         # SSL Configuration
116         ssl_certificate /etc/ssl/certs/selfsigned.crt;
```

```
68 http {
69
70     # HTTPS Server
71     server {
72         listen 443 ssl http2;
73         server_name _;
74
75         # SSL Configuration
76         ssl_certificate /etc/ssl/certs/selfsigned.crt;
77         ssl_certificate_key /etc/ssl/private/selfsigned.key;
78         ssl_protocols TLSv1.2 TLSv1.3;
79         ssl_ciphers HIGH:!aNULL:!MD5;
80         ssl_prefer_server_ciphers on;
81
82         # Security Headers
83         add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
84         add_header X-Frame-Options "SAMEORIGIN" always;
85         add_header X-Content-Type-Options "nosniff" always;
86         add_header X-XSS-Protection "1; mode=block" always;
87
88         # Proxy settings
89         location / {
90             proxy_pass http://backend_servers;
91
92             # Proxy headers
93             proxy_set_header Host $host;
94             proxy_set_header X-Real-IP $remote_addr;
95             proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
96             proxy_set_header X-Forwarded-Proto $scheme;
97         }
98     }
99 }
```

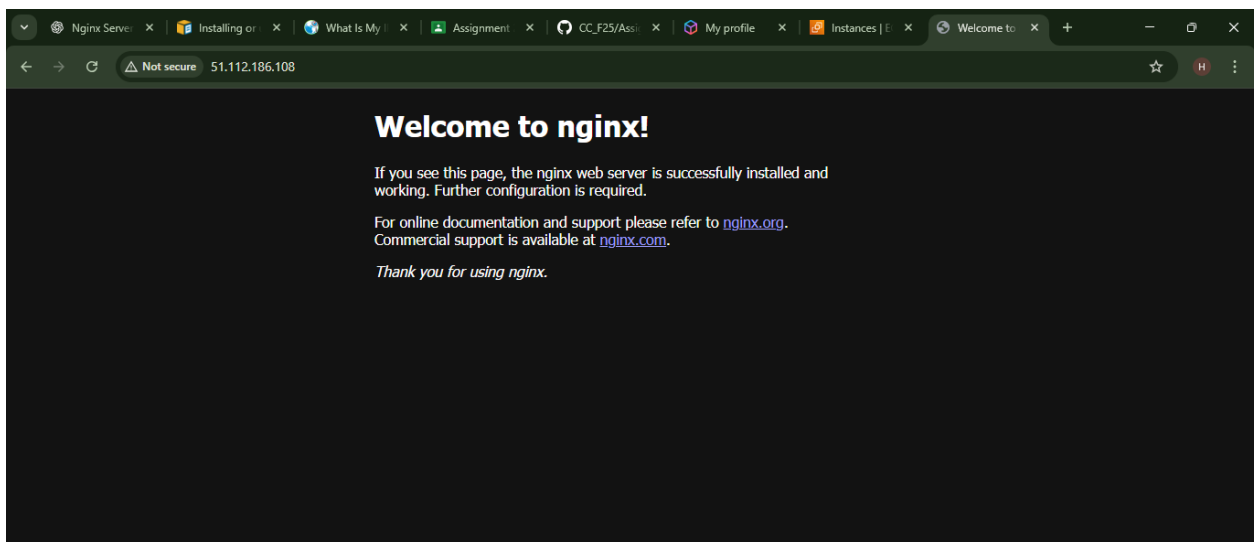
```
scripts > $ nginx-setup.sh
68 http {
69
70     # Cache settings
71     proxy_cache my_cache;
72     proxy_cache_valid 200 60m;
73     proxy_cache_valid 404 10m;
74     proxy_cache_key "$scheme$request_method$host$request_uri";
75     proxy_cache_bypass $http_cache_control;
76     add_header X-Cache-Status $upstream_cache_status;
77
78     # Timeouts
79     proxy_connect_timeout 60s;
80     proxy_send_timeout 60s;
81     proxy_read_timeout 60s;
82 }
83
84 # Health check endpoint
85 location /health {
86     access_log off;
87     return 200 "Nginx is healthy\n";
88     add_header Content-Type text/plain;
89 }
90
91 # HTTP Server (redirect to HTTPS)
92 server {
93     listen 80;
94     server_name _;
95 }
```



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure for a Terraform project, including modules, providers, and scripts. The code editor shows the following content:

```
68 http {
161     server {
165         location / {
167             }
168
169         # Health check endpoint over HTTP
170         location /health {
171             access_log off;
172             return 200 "Nginx is healthy\n";
173             add_header Content-Type text/plain;
174         }
175     }
176 }
177 EOF
178
179 # -----
180 # Step 7: Create cache directory
181 # -----
182 mkdir -p /var/cache/nginx
183 chown -R nginx:nginx /var/cache/nginx
184
185 # -----
186 # Step 8: Test and restart Nginx
187 # -----
188 nginx -t
189 systemctl restart nginx
190
191 echo "Nginx setup completed successfully!"
192 echo "Remember to update backend server IPs in /etc/nginx/nginx.conf"
193
```

- Screenshot: assignment_part3_nginx_default_page.png (before backend configuration)



Part 4: Infrastructure Deployment (15 marks)

4.1 Initial Deployment (5 marks)

Deploy the infrastructure using Terraform.

Tasks:

- Generate SSH key pair if not exists
- Initialize Terraform (terraform init)
- Validate configuration (terraform validate)
- Plan deployment (terraform plan)
- Apply configuration (terraform apply -auto-approve)

Deliverables:

- Screenshot: assignment_part4_ssh_keygen.png

```
PS D:\Assignment2> ssh-keygen -t rsa -b 4096 -f terraform-key
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in terraform-key
Your public key has been saved in terraform-key.pub
The key fingerprint is:
SHA256:Cd1JFpvd4FokyMXLZSPQVnItiIwXiEvnfJxTkP+FnQw admin@Hamail-Alam
The key's randomart image is:
+---[RSA 4096]-----+
|  o B00EB*o |
|  o *=BX*O*.. |
|  . =oo=*=o.. |
|  . o.=+o |
|  .S. |
| |
+-----[SHA256]-----+
PS D:\Assignment2> |
```

- Screenshot: assignment_part4_terraform_init.png

```
PS D:\Assignment2> D:\terraform_1.14.3_windows_amd64\terraform.exe init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/http from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/http v3.5.0
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS D:\Assignment2> |
```

- Screenshot: assignment_part4_terraform_validate.png

```
PS D:\Assignment2> D:\terraform_1.14.3_windows_amd64\terraform.exe validate

Warning: Deprecated attribute

on modules\security\main.tf line 7, in locals:
7:   my_ip_cidr = "${chomp(data.http.my_ip.body)}/32"

The attribute "body" is deprecated. Refer to the provider documentation for details.

Success! The configuration is valid, but there were some validation warnings as shown above.
PS D:\Assignment2> |
```

- Screenshot: assignment_part4_terraform_plan.png

```
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Warning: Deprecated attribute
  on modules\security\main.tf line 7, in locals:
   7:   my_ip_cidr = "${chomp(data.http.my_ip.body)}/32"

The attribute "body" is deprecated. Refer to the provider documentation for details.

PS D:\Assignment2>
```

```
Warning: Deprecated attribute
  on modules\security\main.tf line 7, in locals:
   7:   my_ip_cidr = "${chomp(data.http.my_ip.body)}/32"

The attribute "body" is deprecated. Refer to the provider documentation for details.

(and 2 more similar warnings elsewhere)

Apply complete! Resources: 0 added, 5 changed, 0 destroyed.

Outputs:

backend_sg_id = "sg-08e027917340b1443"
nginx_sg_id = "sg-0addbdc83a4f35e04"
PS D:\Assignment2>
```

- Screenshot: assignment_part4_terraform_apply.png (showing all 4 instances created)

```
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Warning: Deprecated attribute
  on modules\security\main.tf line 7, in locals:
   7:   my_ip_cidr = "${chomp(data.http.my_ip.body)}/32"

The attribute "body" is deprecated. Refer to the provider documentation for details.

(and 2 more similar warnings elsewhere)

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

backend_sg_id = "sg-08e027917340b1443"
nginx_sg_id = "sg-0addbdc83a4f35e04"
PS D:\Assignment2>
```

4.2 Output Configuration (5 marks)

Create comprehensive outputs in outputs.tf:

Networking Outputs

```
output "vpc_id" {
  description = "VPC ID"
  value       = module.networking.vpc_id
}
```

```
output "subnet_id" {
  description = "Subnet ID"
  value       = module.networking.subnet_id
}
```

Nginx Server Outputs

```
output "nginx_public_ip" {
  description = "Nginx server public IP"
  value       = module.nginx_server.public_ip
}
```

```
output "nginx_instance_id" {
  description = "Nginx server instance ID"
  value      = module.nginx_server.instance_id
}
```

Backend Server Outputs

```
output "backend_servers_info" {
  description = "Backend servers information"
  value = {
    for name, server in module.backend_servers : name => {
      instance_id = server.instance_id
      public_ip   = server.public_ip
      private_ip  = server.private_ip
    }
  }
}
```

Quick Configuration Guide

```
output "configuration_guide" {
  value = <<-EOT
```

```
=====
DEPLOYMENT SUCCESSFUL!
=====
```

Next Steps:

1. SSH into Nginx server: `ssh ec2-user@${module.nginx_server.public_ip}`
2. Edit Nginx config: `sudo vim /etc/nginx/nginx.conf`
3. Update backend IPs in upstream block:
 - `BACKEND_IP_1: ${module.backend_servers["web-1"].private_ip}`
 - `BACKEND_IP_2: ${module.backend_servers["web-2"].private_ip}`
 - `BACKEND_IP_3: ${module.backend_servers["web-3"].private_ip}`
4. Restart Nginx: `sudo systemctl restart nginx`
5. Test: `https://${module.nginx_server.public_ip}`

Backend Servers:

```
${join("\n  ", [for name, server in module.backend_servers : "- ${name}:
${server.public_ip} (private: ${server.private_ip})"]])}
```

```
=====
EOT
```

```
}
```

Tasks:

- Create all required outputs
- Add helpful descriptions
- Include configuration guide
- Display outputs after apply

Commands:

`terraform output`

`terraform output -json > outputs.json`

Deliverables:

- Screenshot: assignment_part4_terraform_output.png

```
=====
DEPLOYMENT SUCCESSFUL!
=====

Next Steps:

1. SSH into Nginx server:
  ssh ec2-user@51.112.186.108

2. Edit Nginx configuration:
  sudo vim /etc/nginx/nginx.conf

3. Update backend private IPs in upstream block:
  - web-1
  - web-2
  - web-3

4. Restart Nginx:
  sudo systemctl restart nginx

5. Open browser:
  https:51.112.186.108

=====

EOT
nginx_sg_id = "sg-0addbdc83a4f35e04"
PS D:\Assignment2>
```

- Screenshot: assignment_part4_outputs_json.png

```
PS D:\Assignment2> D:\terraform_1.14.3_windows_amd64\terraform.exe output -json > outputs.json
PS D:\Assignment2> cat outputs.json
{
  "backend_sg_id": {
    "sensitive": false,
    "type": "string",
    "value": "sg-08e027917340b1443"
  },
  "configuration_guide": {
    "sensitive": false,
    "type": "string",
    "value": "\r\n=====
\r\n\r\nNext Steps:\r\n\r\n1. SSH into Nginx server:\r\n  ssh ec2-user@51.112.186.108\r\n\r\n2. Edit Nginx configuration:\r\n  sudo vim /etc/nginx/nginx.conf\r\n\r\n3. Update backend private IPs
in upstream block:\r\n  - web-1\r\n  - web-2\r\n  - web-3\r\n\r\n4. Restart Nginx:\r\n  sudo systemctl
restart nginx\r\n\r\n5. Open browser:\r\n  https:51.112.186.108\r\n\r\n=====
\r\n"
  },
  "nginx_sg_id": {
    "sensitive": false,
    "type": "string",
    "value": "sg-0addbdc83a4f35e04"
  }
}
PS D:\Assignment2> |
```

4.3 AWS Console Verification (5 marks)

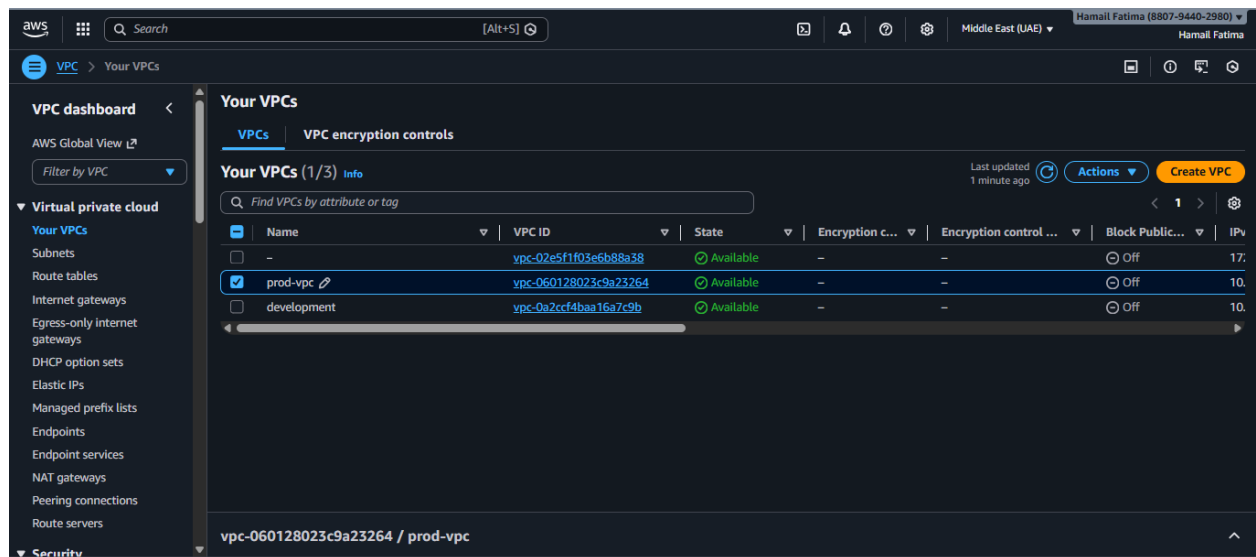
Verify all resources in AWS Console.

Tasks:

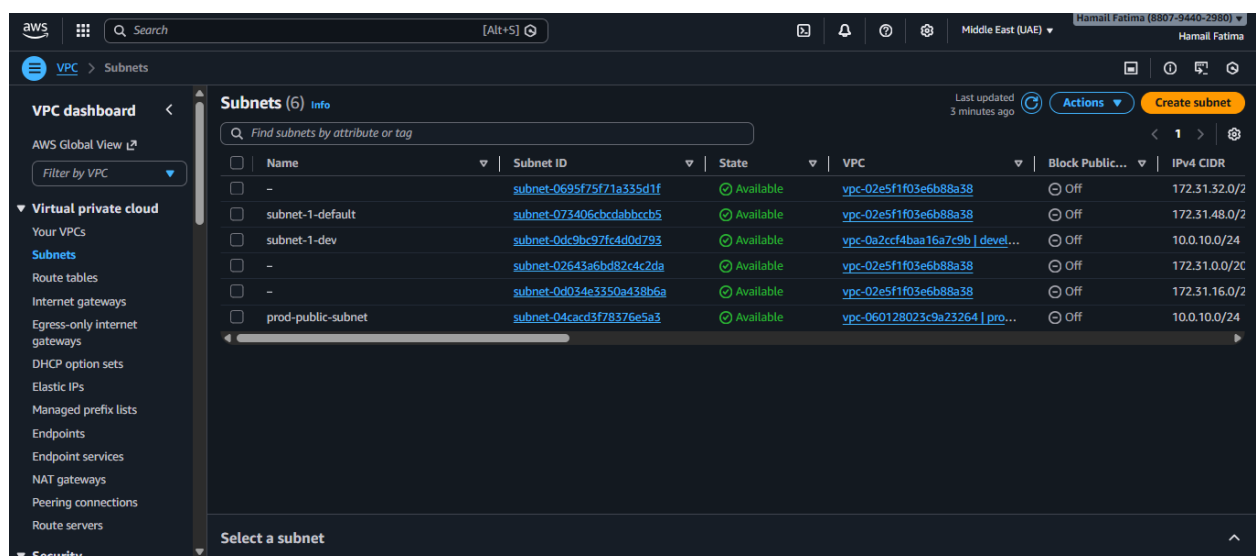
- Verify VPC created
- Verify Subnet created
- Verify Internet Gateway attached
- Verify Route Table configured
- Verify Security Groups created with correct rules
- Verify all 4 EC2 instances running
- Verify Key Pairs created

Deliverables:

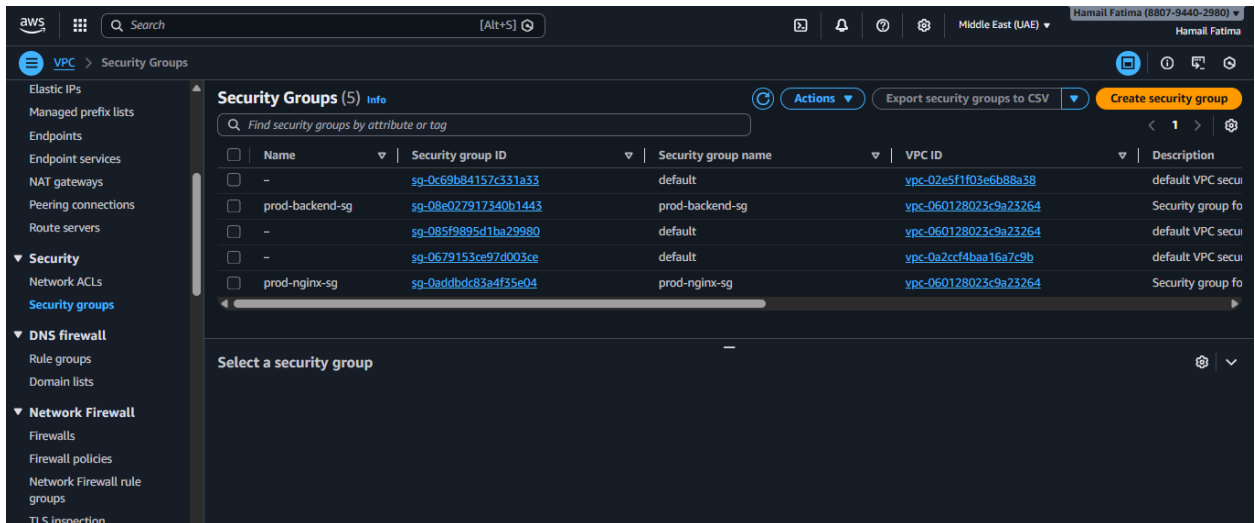
- Screenshot: assignment_part4_aws_vpc. png (VPC console)



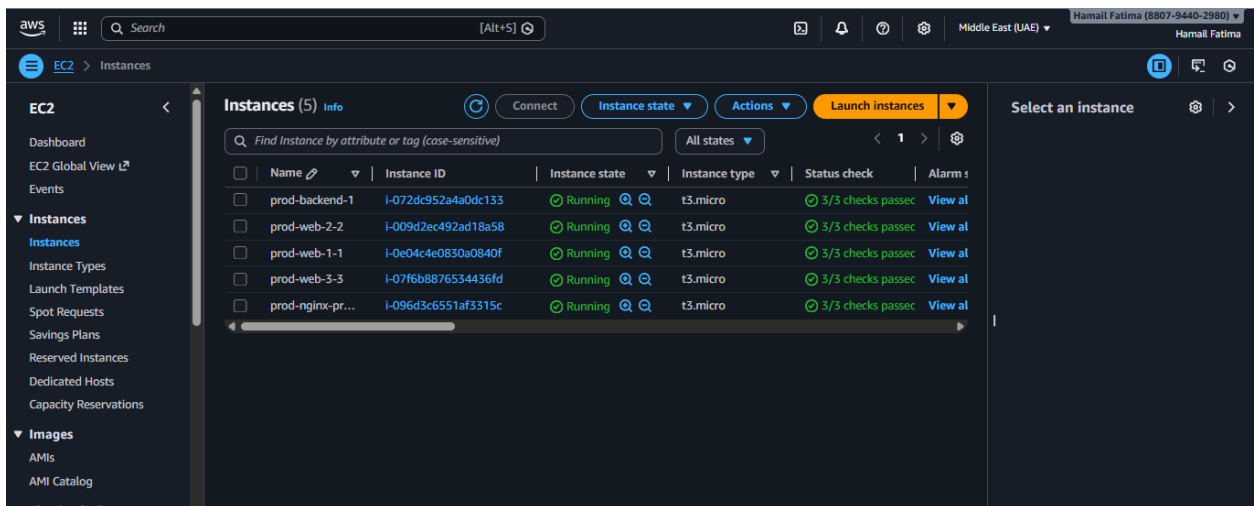
- Screenshot: assignment_part4_aws_subnet.png (Subnet details)



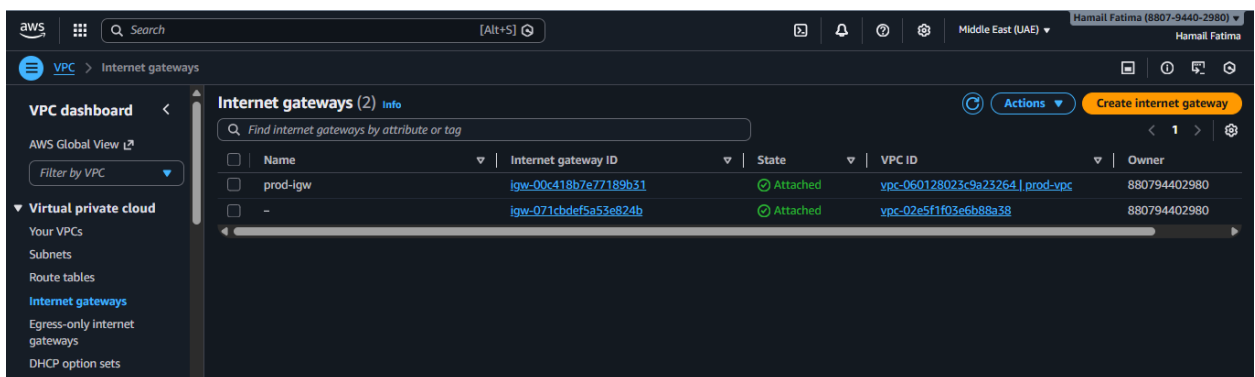
- Screenshot: assignment_part4_aws_security_groups.png (Both security groups)



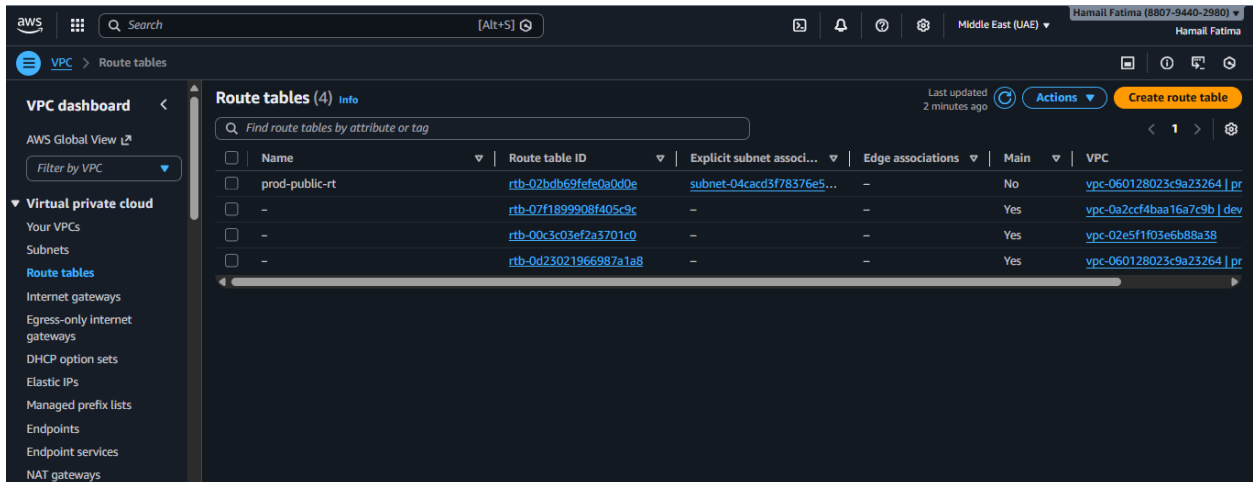
- Screenshot: assignment_part4_aws_instances.png (All 4 running instances)



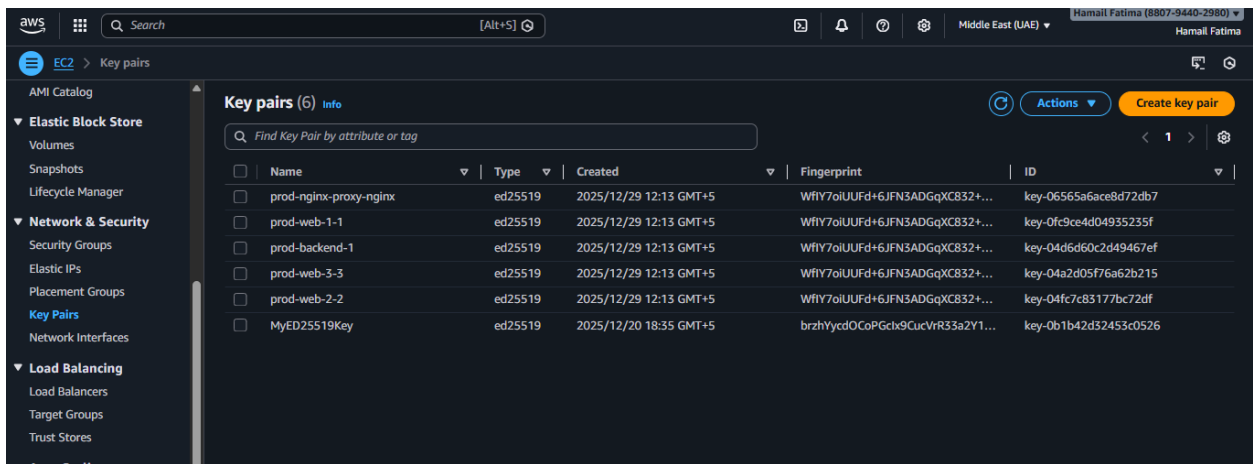
Verify Internet Gateway



Verify Route Table



Verify Key Pair



Part 5: Nginx Configuration & Testing (25 marks)

5.1 Update Nginx Backend Configuration (5 marks)

SSH into the Nginx server and update the configuration with actual backend IPs.

Tasks:

- SSH into Nginx server
- Edit /etc/nginx/nginx.conf
- Replace placeholder IPs with actual private IPs of backend servers
- Test Nginx configuration
- Restart Nginx service

Commands:

```
# SSH into Nginx server
ssh ec2-user@<nginx-public-ip>
```

```
# Edit configuration
sudo vim /etc/nginx/nginx.conf
```

```
# Update upstream block:
upstream backend_servers {
    server <web-1-private-ip>:80;
    server <web-2-private-ip>:80;
    server <web-3-private-ip>:80 backup;
}
```

```
# Test configuration
sudo nginx -t
```

```
# Restart Nginx
sudo systemctl restart nginx
```

```
# Check status
sudo systemctl status nginx
```

Deliverables:

- Screenshot: assignment_part5_ssh_nginx.png (SSH session)

```
PS D:\Assignment2> ssh -i terraform-key ec2-user@158.252.77.217  
The authenticity of host '158.252.77.217 (158.252.77.217)' can't be established.  
ED25519 key fingerprint is SHA256:0pJk59wJ6MDRLEACIMBXNjtT4EYGWKGQbkKhSxjumR4o.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '158.252.77.217' (ED25519) to the list of known hosts.
```

```
#_
```

```
~\##### Amazon Linux 2
```

```
n\n\#####\ AL2 End of Life is 2026-06-30.
```

```
n\n|###|\
```

```
n\n#\V n \-->
```

```
n\n\n A newer version of Amazon Linux is available!
```

```
n\n.-.\n./\ Amazon Linux 2023, GA and supported until 2028-03-15.
```

```
_m'\_\ https://aws.amazon.com/linux/amazon-linux-2023/
```

- Screenshot: assignment_part5_nginx_conf_updated.png (updated upstream block)

```
ec2-user@ip-10-0-10-118:~  
root /usr/share/nginx/html;  
  
# Load configuration files for the default server block.  
include /etc/nginx/default.d/*.conf;  
  
error_page 404 /404.html;  
location = /404.html {  
}  
  
error_page 500 502 503 504 /50x.html;  
location = /50x.html {  
}  
}  
  
upstream backend_servers {  
    server 158.252.77.217:80;  
}  
  
# Settings for a TLS enabled server.  
#  
# server {  
#     listen      443 ssl;  
#     listen      [::]:443 ssl;  
#     http2        on;  
#     server_name  _;  
#     root         /usr/share/nginx/html;  
#  
#     ssl_certificate "/etc/pki/nginx/server.crt";  
#     ssl_certificate_key "/etc/pki/nginx/private/server.key";  
#     ssl_session_cache shared:SSL:1m;  
# }  
"/etc/nginx/nginx.conf" 87L, 2400B
```

- Screenshot: assignment_part5_nginx_test.png (nginx -t output)

```
[ec2-user@ip-10-0-10-118 ~]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-10-0-10-118 ~]$
```

- Screenshot: assignment_part5_nginx_restart.png (restart and status)

```
[ec2-user@ip-10-0-10-118 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-118 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; vendor preset: disabled)
   Active: active (running) since Mon 2025-12-29 20:31:47 UTC; 48ms ago
     Process: 2636 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
     Process: 2632 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 2630 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
    Main PID: 2638 (nginx)
   CGroup: /system.slice/nginx.service
           └─2638 nginx: master process /usr/sbin/nginx
             └─2639 nginx: worker process
               └─2640 nginx: worker process

Dec 29 20:31:47 ip-10-0-10-118.me-central-1.compute.internal systemd[1]: Starting The nginx HTTP and reverse proxy ....
Dec 29 20:31:47 ip-10-0-10-118.me-central-1.compute.internal nginx[2632]: nginx: the configuration file /etc/nginx/...ok
Dec 29 20:31:47 ip-10-0-10-118.me-central-1.compute.internal nginx[2632]: nginx: configuration file /etc/nginx/nginx...ul
Dec 29 20:31:47 ip-10-0-10-118.me-central-1.compute.internal systemd[1]: Started The nginx HTTP and reverse proxy s...r.
Hint: Some lines were ellipsized, use -l to show in full.
[ec2-user@ip-10-0-10-118 ~]$
```

5.2 Test Load Balancing (5 marks)

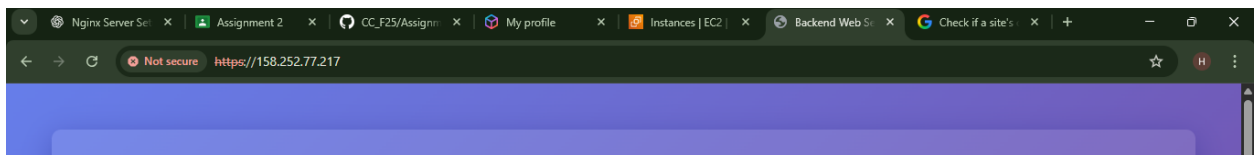
Test that Nginx is properly load balancing between web-1 and web-2.

Tasks:

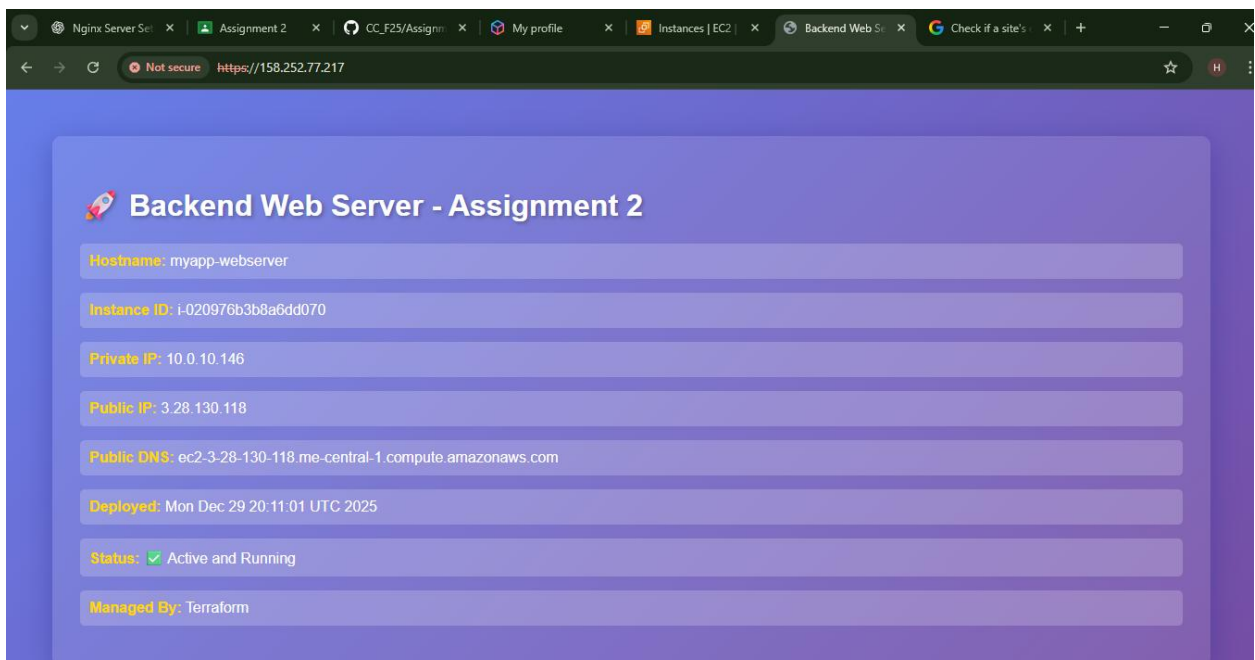
- Open browser to <https://<nginx-public-ip>>
- Accept the security warning for self-signed certificate
- Reload page multiple times (at least 10 times)
- Verify traffic alternates between web-1 and web-2
- Verify web-3 is NOT serving traffic (it's backup only)
- Document the load balancing pattern

Deliverables:

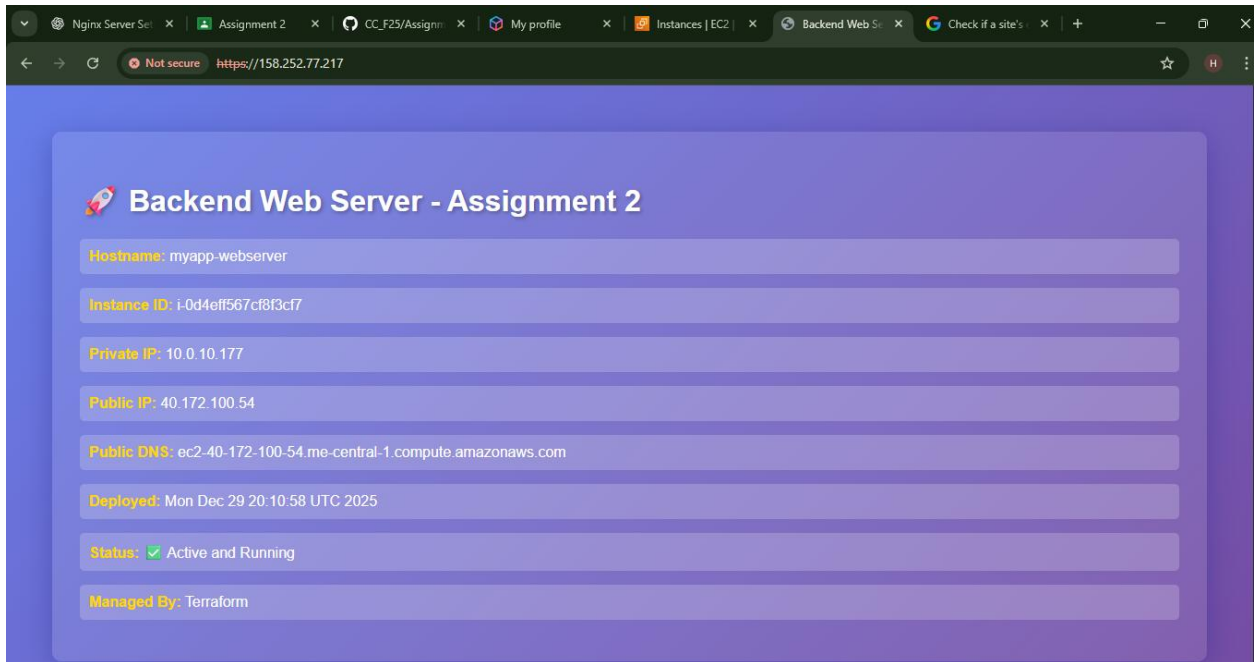
- Screenshot: assignment_part5_ssl_warning.png (browser security warning)



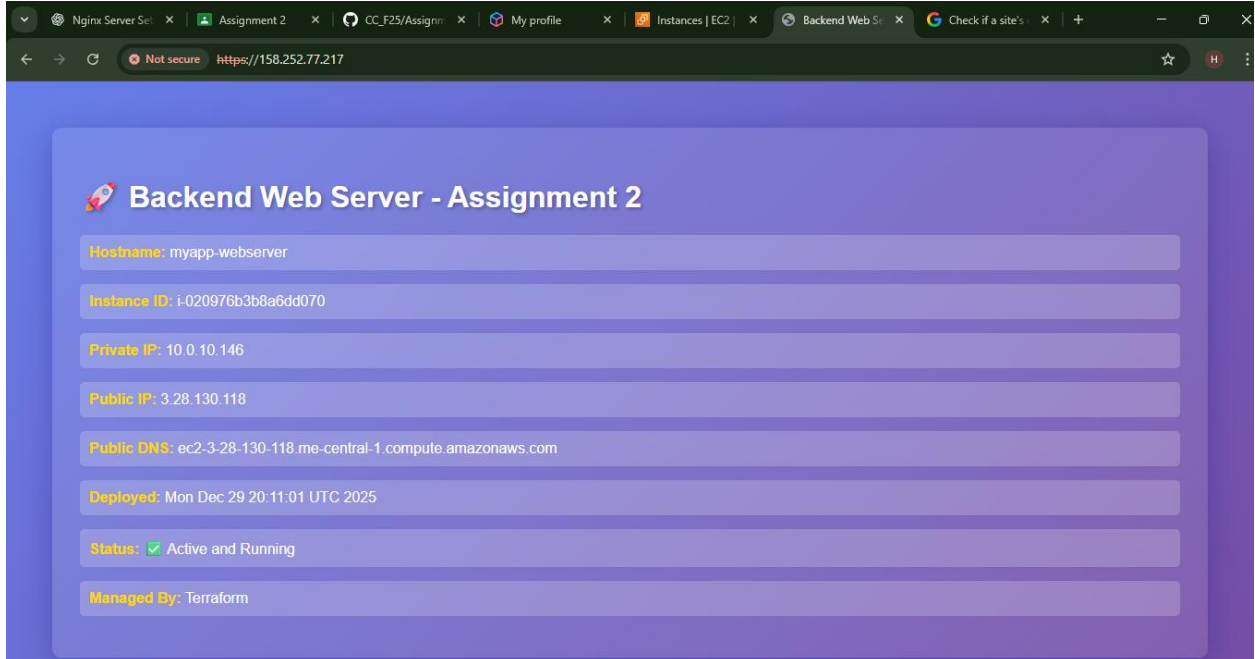
- Screenshot: assignment_part5_web1_response.png (showing web-1 content)



- Screenshot: assignment_part5_web2_response. png (showing web-2 content)



- Screenshot: assignment_part5_load_balancing_demo.png (multiple reloads showing alternation)



5.3 Test Cache Functionality (5 marks)

Verify that Nginx caching is working correctly.

Tasks:

- Open browser developer tools (F12)
- Navigate to Network tab
- Clear browser cache
- Load `https://<nginx-public-ip>`
- Check response headers for `X-Cache-Status: MISS` (first request)
- Reload page
- Check response headers for `X-Cache-Status: HIT` (cached request)
- Verify cache directory on Nginx server

Commands on Nginx server:

Check cache directory

```
ls -la /var/cache/nginx/
```

Monitor access logs

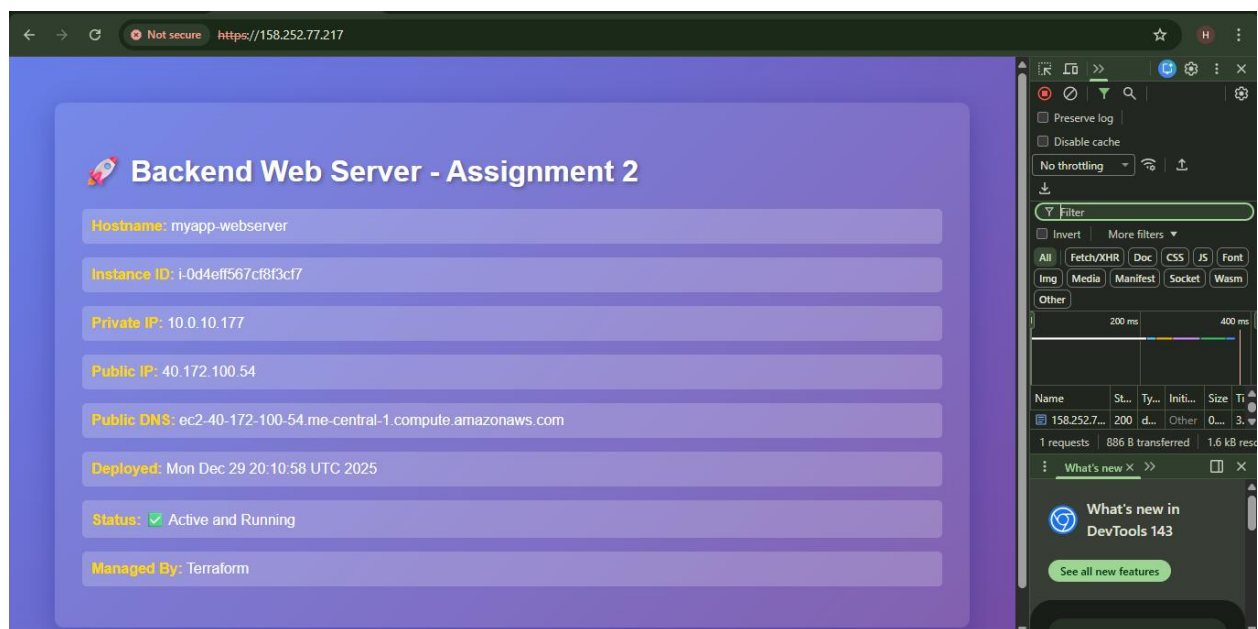
```
sudo tail -f /var/log/nginx/access.log
```

Check cache status in logs

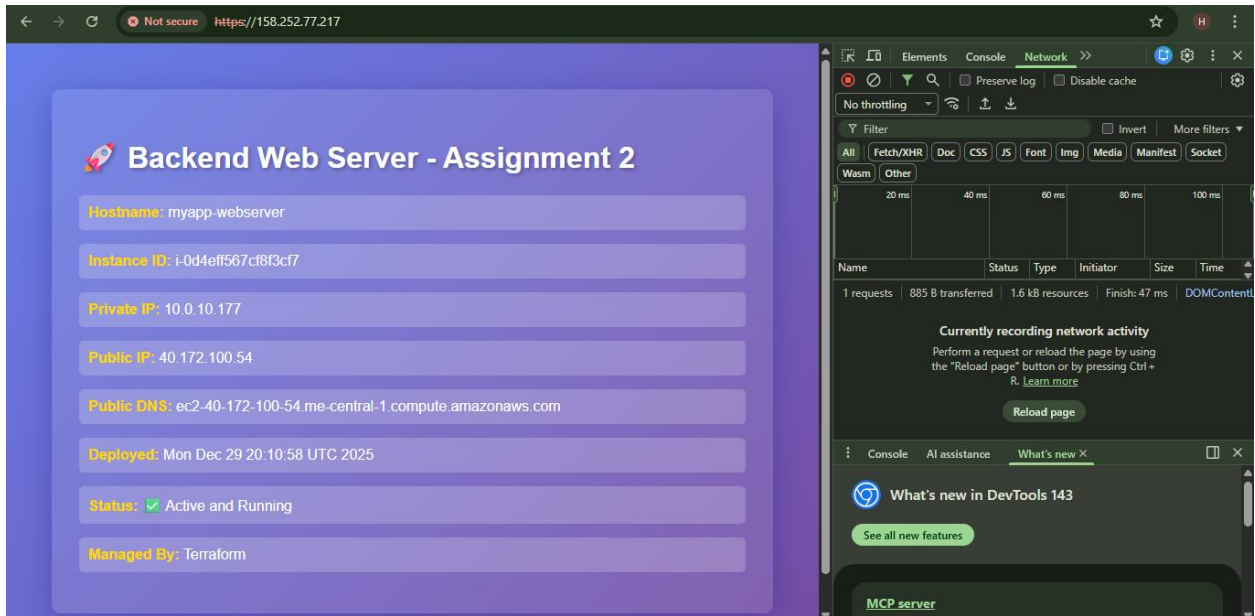
```
sudo grep "Cache:" /var/log/nginx/access.log | tail -20
```

Deliverables:

- Screenshot: `assignment_part5_cache_miss.png` (first request - MISS)



- Screenshot: `assignment_part5_cache_hit.png` (second request - HIT)



- Screenshot: assignment_part5_cache_directory.png (cache folder contents)

```
[ec2-user@ip-10-0-10-118 ~]$ sudo ls -la /var/cache/nginx/
total 0
drwx----- 4 nginx root 24 Dec 29 21:21 .
drwxr-xr-x 7 root root 76 Dec 29 21:19 ..
drwx----- 3 nginx nginx 16 Dec 29 21:21 1
drwx----- 3 nginx nginx 16 Dec 29 21:21 2
[ec2-user@ip-10-0-10-118 ~]$
```

- Screenshot: assignment_part5_access_log_cache.png (access log showing cache status)

```
[ec2-user@ip-10-0-10-118 ~]$ sudo grep "Cache:" /var/log/nginx/access.log | tail -20
103.149.240.116 - - [29/Dec/2025:21:40:34 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:40:34 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:40:34 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:40:35 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
195.178.110.230 - - [29/Dec/2025:21:43:56 +0000] "CONNECT www.google.com:443 HTTP/1.1" 400 157 "-" "-" "-" Cache: -
103.149.240.116 - - [29/Dec/2025:21:45:00 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:01 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:02 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:02 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:03 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:04 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:04 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:05 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:06 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:06 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:07 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:07 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
```

5.4 Test High Availability (Backup Server) (5 marks)

Test the backup server functionality by simulating primary server failure.

Tasks:

- SSH into web-1 and stop Apache
- Reload Nginx page - should show web-2 only
- SSH into web-2 and stop Apache

- Reload Nginx page - should now show web-3 (backup activated)
- Restart web-1 and web-2
- Verify traffic returns to web-1 and web-2

Commands:

On web-1

```
ssh ec2-user@<web-1-public-ip>
sudo systemctl stop httpd
sudo systemctl status httpd
```

On web-2

```
ssh ec2-user@<web-2-public-ip>
sudo systemctl stop httpd
```

Verify on Nginx

```
ssh ec2-user@<nginx-public-ip>
sudo tail -f /var/log/nginx/error.log
```

Restart services

```
sudo systemctl start httpd
```

Deliverables:

- Screenshot: assignment_part5_web1_stopped.png (web-1 Apache stopped)

```
ED25519 key fingerprint is SHA256:XlFX7nPxcqh0xWJ5EboSe4pDVUAJl+afUvEOxpJLFg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '40.172.100.54' (ED25519) to the list of known hosts.

#_
##### Amazon Linux 2
#####\
#####| AL2 End of Life is 2026-06-30.
#####| \##|
#####| \#/
#####| V# ' ->
#####| A newer version of Amazon Linux is available!
#####|
#####| Amazon Linux 2023, GA and supported until 2028-03-15.
#####| https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@myapp-webserver ~]$ sudo systemctl stop httpd
[ec2-user@myapp-webserver ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: inactive (dead) since Mon 2025-12-29 21:52:27 UTC; 9s ago
     Docs: man:httpd.service(8)
  Process: 2433 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
 Main PID: 2433 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 75; Idle/Busy workers 100/0; Requests/sec: 0.0123; Bytes served/sec: 32 B/sec"

Dec 29 20:10:58 ip-10-0-10-177.me-central-1.compute.internal systemd[1]: Starting The Apache HTTP Server...
Dec 29 20:10:58 ip-10-0-10-177.me-central-1.compute.internal systemd[1]: Started The Apache HTTP Server.
Dec 29 21:52:26 myapp-webserver systemd[1]: Stopping The Apache HTTP Server...
Dec 29 21:52:27 myapp-webserver systemd[1]: Stopped The Apache HTTP Server.
[ec2-user@myapp-webserver ~]$
```

- Screenshot: assignment_part5_web2_stopped.png (web-2 Apache stopped)


```
2025/12/29 21:14:44 [warn] 3013#3013: the "listen ... http2" directive is deprecated, use the "http2" directive instead
in /etc/nginx/nginx.conf:47
2025/12/29 21:14:44 [emerg] 3013#3013: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (S
SL: error:02001002:system library:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D
080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:19:25 [warn] 3039#3039: the "listen ... http2" directive is deprecated, use the "http2" directive instead
in /etc/nginx/nginx.conf:47
2025/12/29 21:19:38 [warn] 3047#3047: the "listen ... http2" directive is deprecated, use the "http2" directive instead
in /etc/nginx/nginx.conf:47
2025/12/29 21:19:38 [warn] 3050#3050: the "listen ... http2" directive is deprecated, use the "http2" directive instead
in /etc/nginx/nginx.conf:47
2025/12/29 21:48:28 [warn] 3212#3212: the "listen ... http2" directive is deprecated, use the "http2" directive instead
in /etc/nginx/nginx.conf:47
2025/12/29 21:48:28 [warn] 3223#3223: the "listen ... http2" directive is deprecated, use the "http2" directive instead
in /etc/nginx/nginx.conf:47
2025/12/29 21:48:28 [warn] 3227#3227: the "listen ... http2" directive is deprecated, use the "http2" directive instead
in /etc/nginx/nginx.conf:47
2025/12/29 21:55:27 [error] 3231#3231: *13 connect() failed (111: Connection refused) while connecting to upstream, cli
nt: 103.149.240.116, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.177:80/", host: "158.252.77.217"
2025/12/29 22:00:22 [error] 3231#3231: *42 connect() failed (111: Connection refused) while connecting to upstream, cli
nt: 103.149.240.116, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.146:80/", host: "158.252.77.217"
2025/12/29 22:00:22 [error] 3231#3231: *42 connect() failed (111: Connection refused) while connecting to upstream, cli
nt: 103.149.240.116, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.177:80/", host: "158.252.77.217"
```

- Screenshot: assignment_part5_services_restored.png (all services back online)

```
PS C:\Users\admin> cd D:\Assignment2
PS D:\Assignment2> ssh -i terraform-key ec2-user@40.172.100.54
Last login: Mon Dec 29 21:52:16 2025 from 103.149.240.116

#_
##### Amazon Linux 2
#####
#####| AL2 End of Life is 2026-06-30.
#####|
#####|/
##### V__> A newer version of Amazon Linux is available!
#####
#####
##### Amazon Linux 2023, GA and supported until 2028-03-15.
##### https://aws.amazon.com/linux/amazon-linux-2023/

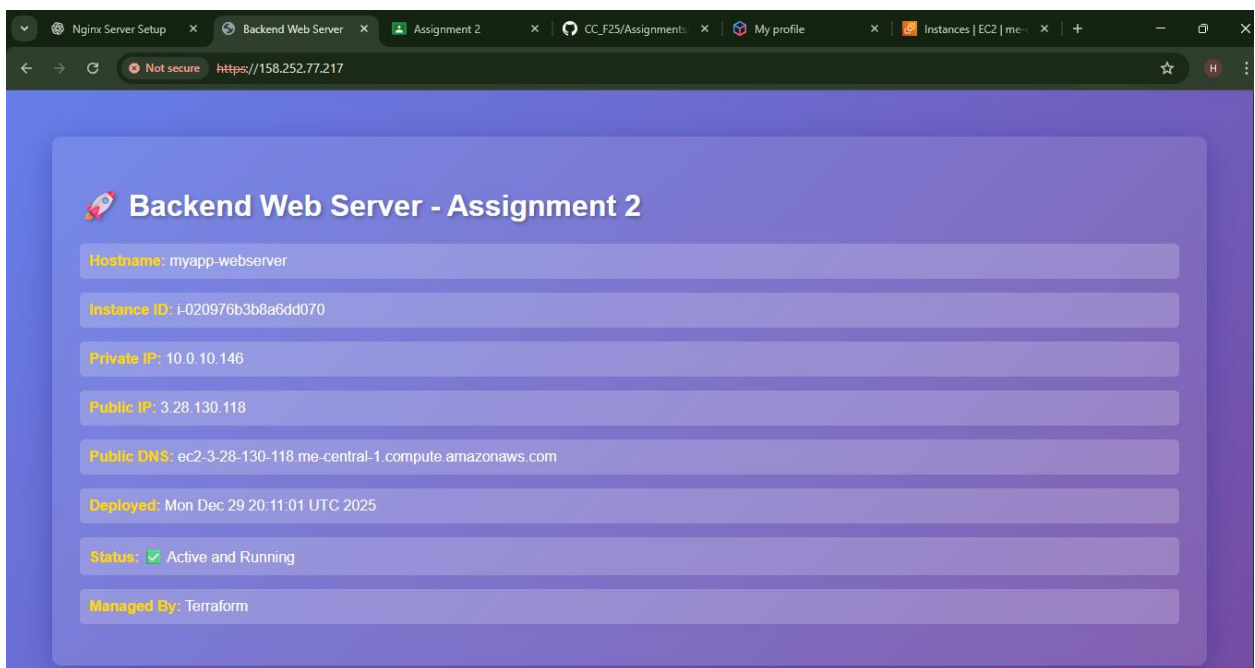
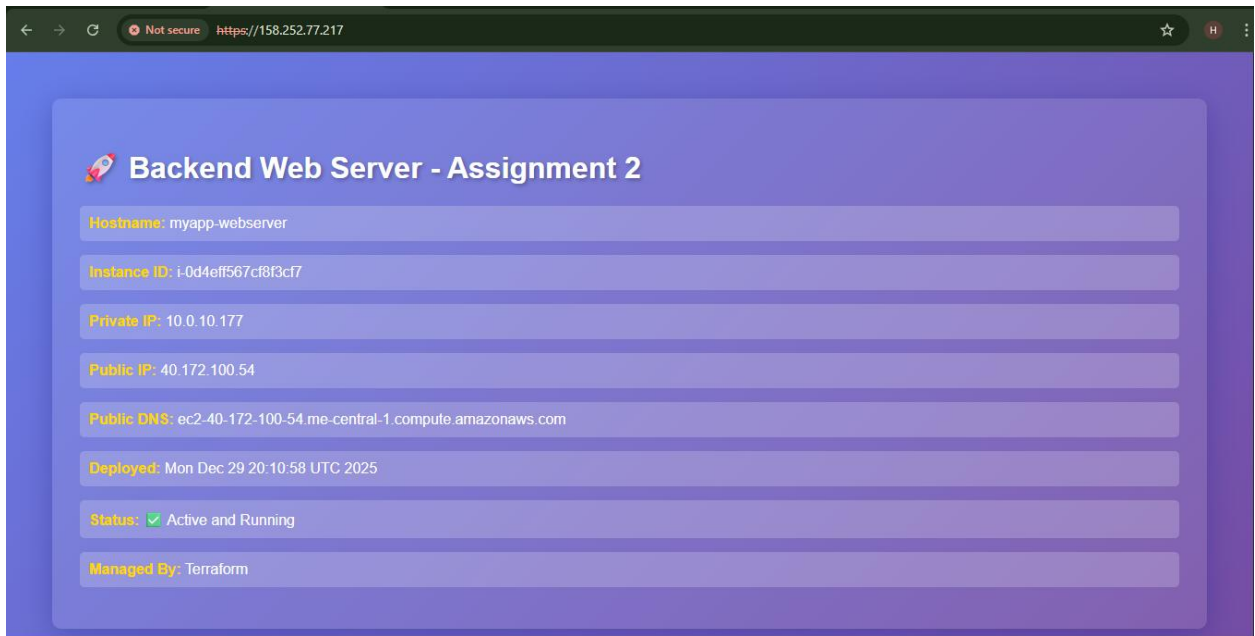
[ec2-user@myapp-webserver ~]$ sudo systemctl start httpd
[ec2-user@myapp-webserver ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: active (running) since Mon 2025-12-29 22:02:26 UTC; 3s ago
     Docs: man:httpd.service(8)
 Main PID: 31898 (httpd)
    Status: "Processing requests..."
    CGroup: /system.slice/httpd.service
            └─31898 /usr/sbin/httpd -DFOREGROUND
              └─31899 /usr/sbin/httpd -DFOREGROUND
                └─31900 /usr/sbin/httpd -DFOREGROUND
                  └─31919 /usr/sbin/httpd -DFOREGROUND
                    └─31924 /usr/sbin/httpd -DFOREGROUND
                      └─31929 /usr/sbin/httpd -DFOREGROUND

PS D:\Assignment2> ssh -i terraform-key ec2-user@3.28.130.118
Last login: Mon Dec 29 21:57:48 2025 from 103.149.240.116

#_
##### Amazon Linux 2
#####
#####| AL2 End of Life is 2026-06-30.
#####|
#####|/
##### V__> A newer version of Amazon Linux is available!
#####
#####
##### Amazon Linux 2023, GA and supported until 2028-03-15.
##### https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@myapp-webserver ~]$ sudo systemctl start httpd
[ec2-user@myapp-webserver ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: active (running) since Mon 2025-12-29 22:03:26 UTC; 2s ago
     Docs: man:httpd.service(8)
 Main PID: 31819 (httpd)
    Status: "Processing requests..."
    CGroup: /system.slice/httpd.service
            └─31819 /usr/sbin/httpd -DFOREGROUND
              └─31820 /usr/sbin/httpd -DFOREGROUND
                └─31821 /usr/sbin/httpd -DFOREGROUND
                  └─31822 /usr/sbin/httpd -DFOREGROUND
                    └─31823 /usr/sbin/httpd -DFOREGROUND
                      └─31824 /usr/sbin/httpd -DFOREGROUND

Dec 29 22:03:25 myapp-webserver systemd[1]: Starting The Apache HTTP Server...
Dec 29 22:03:26 myapp-webserver httpd[31819]: AH00558: error: Could not reliably determine the server's fully qualified domain name: please consider the parameter 'ServerName' under the '<VirtualHost>' section of your configuration file to set the 'ServerName' directive explicitly to the FQDN of the machine on which the server runs. See https://httpd.apache.org/docs/2.4/vhosts/other.html#servername for more information.
Dec 29 22:03:26 myapp-webserver systemd[1]: Started The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
```



5.5 Security & Performance Analysis (5 marks)

Analyze the security headers and performance of your Nginx setup.

Tasks:

- Check SSL/TLS certificate details
- Verify security headers in response
- Test HTTP to HTTPS redirect
- Check response times
- Analyze Nginx logs

Tools & Commands:

Check certificate

```
openssl s_client -connect <nginx-ip>:443 -showcerts
```

On Nginx server - view certificate

```
sudo openssl x509 -in /etc/ssl/certs/selfsigned.crt -text -noout
```

```
# Check security headers (from local machine)
curl -I -k https://<nginx-public-ip>
```

```
# Test HTTP redirect
curl -I http://<nginx-public-ip>
```

```
# Monitor error logs
sudo tail -50 /var/log/nginx/error.log
```

```
# Monitor access logs
sudo tail -50 /var/log/nginx/access.log
```

```
# Check Nginx worker processes
ps aux | grep nginx
```

Deliverables:

- Screenshot: assignment_part5_ssl_certificate.png (certificate details)

```
New, TLSv1/SSLv3, Cipher is ECDHE-RSA-AES256-GCM-SHA384
Server public key is 2048 bit
Secure Renegotiation IS supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
  Protocol  : TLSv1.2
  Cipher    : ECDHE-RSA-AES256-GCM-SHA384
  Session-ID: 07897B57D157D49BEFDD2B5D944B36933A9076CA50ED0BF82BC6C4F1D81B82B
  Session-ID-ctx:
  Master-Key: 8176EEF113B48F5A688BC9374A0666B1A5FBF94864CAD5DED9F922A0C45E25C389FBE6EA29555079BA8CE80830C07751
  Key-Arg   : None
  Krb5 Principal: None
  PSK identity: None
  PSK identity hint: None
  TLS session ticket lifetime hint: 300 (seconds)
  TLS session ticket:
0000 - 9c d1 ff 3b a5 7d fd 29-c8 ea 74 e2 80 c9 ee ae ...;}.).t....
0010 - 0e fa 33 80 8f be 88 7c-63 9e 1e 92 09 58 27 29 ..3....|c....X')
0020 - 42 18 b3 61 47 dd 39 d0-1c c2 36 e6 d1 13 2e 61 B..aG.9...6....a
0030 - df f7 a6 77 c2 5f f5 ba-e2 ff 02 b2 65 58 31 87 ...w.....eXl.
0040 - 0a e5 7e fc b2 ec 6c 09-fc db d9 cd e4 3c b0 70 ..~...l.....<.p
0050 - cb e1 45 20 2b da 80 61-6b f5 71 c7 a0 4c 24 34 ..E +..ak.q..L$4
0060 - 61 90 aa d7 66 96 17 13-34 a5 9d bd c4 1c fb ed a...f...4.....
0070 - 4c de 03 cf 5b a6 23 c3-6a 8d b0 4f f3 3f 1d 20 L...[.#.j..0.?.
0080 - b4 17 81 b9 b8 58 5d c5-c1 4d e9 82 d5 74 7c 42 .....X]..M...t|B
0090 - 6b f9 6a 54 56 5d f4 06-59 6b 52 b0 2a 7b 0b 5a k.jTV]..YkR.*{.Z
00a0 - bc ef e1 9a 4b d0 07 d1-e4 be 30 84 2c 65 06 8f ....K.....0.,e..

Start Time: 1767046230
Timeout    : 300 (sec)
Verify return code: 18 (self signed certificate)
---
```

- Screenshot: assignment_part5_security_headers.png (response headers showing security headers)

```
[ec2-user@ip-10-0-10-118 ~]$ curl -I -k https://158.252.77.217
HTTP/2 200
server: nginx/1.28.0
date: Mon, 29 Dec 2025 22:12:47 GMT
content-type: text/html; charset=UTF-8
content-length: 1584
vary: Accept-Encoding
curl: (92) HTTP/2 stream 1 was not closed cleanly: PROTOCOL_ERROR (err 1)
[ec2-user@ip-10-0-10-118 ~]$
```

- Screenshot: assignment_part5_http_redirect.png (301 redirect test)

```
[ec2-user@ip-10-0-10-118 ~]$ curl -I http://158.252.77.217
HTTP/1.1 301 Moved Permanently
Server: nginx/1.28.0
Date: Mon, 29 Dec 2025 22:13:56 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://158.252.77.217/

[ec2-user@ip-10-0-10-118 ~]$
```

- Screenshot: assignment_part5_error_log_analysis.png (error log review)

```

rec2-user@ip-10-0-10-118 ~$ sudo tail -50 /var/log/nginx/error.log
2025/12/29 28:28:42 [error] 2461#2461: *1 open() "/usr/share/nginx/html/.git/config" failed (2: No such file or directory), client: 195.178.110.162,
server: _, request: "GET /.git/config HTTP/1.1", host: "158.252.77.217"
2025/12/29 28:46:39 [error] 2781#2781: cannot load certificate "/etc/pki/nginx/server.crt": BIO_new_file() failed (SSL: error:02001002:system librar
y:fopen:No such file or directory:fopen('/etc/pki/nginx/server.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 28:48:46 [error] 2790#2790: "listen" directive is not allowed here in /etc/nginx/nginx.conf:61
2025/12/29 28:51:26 [warn] 2805#2805: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:62
2025/12/29 28:51:26 [error] 2805#2805: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 28:53:53 [warn] 2822#2822: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:62
2025/12/29 28:53:53 [error] 2822#2822: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 28:53:53 [warn] 2833#2833: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:62
2025/12/29 28:53:53 [error] 2833#2833: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 28:56:15 [warn] 2857#2857: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:62
2025/12/29 28:56:15 [error] 2857#2857: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:09:55 [warn] 2968#2968: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:09:55 [error] 2968#2968: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:09:55 [warn] 2976#2976: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:09:55 [error] 2976#2976: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:12:58 [warn] 3004#3004: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:12:58 [error] 3004#3004: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:14:44 [warn] 3013#3013: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:14:44 [error] 3013#3013: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:19:25 [warn] 3039#3039: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:19:38 [warn] 3047#3047: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47

```

- Screenshot: assignment_part5_access_log_analysis.png (access log patterns)

```
[ec2-user@ip-10-0-10-118 ~]$ sudo tail -50 /var/log/nginx/error.log
2025/12/29 20:28:42 [error] 2461#2461: *1 open() "/usr/share/nginx/html/.git/config" failed (2: No such file or directory), client: 195.178.110.162,
server: , request: "GET /git/config HTTP/1.1", host: "158.252.77.217"
2025/12/29 20:46:39 [emerg] 2781#2781: cannot load certificate "/etc/pki/nginx/server.crt": BIO_new_file() failed (SSL: error:02001002:system librar
y:fopen:No such file or directory:fopen('/etc/pki/nginx/server.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 20:48:46 [emerg] 2790#2790: "listen" directive is not allowed here in /etc/nginx/nginx.conf:61
2025/12/29 20:51:26 [warn] 2805#2805: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:62
2025/12/29 20:51:26 [error] 2805#2805: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 20:53:53 [warn] 2822#2822: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:62
2025/12/29 20:53:53 [error] 2822#2822: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 20:53:53 [warn] 2833#2833: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:62
2025/12/29 20:53:53 [error] 2833#2833: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 20:56:15 [warn] 2857#2857: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:62
2025/12/29 20:56:15 [error] 2857#2857: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:09:55 [warn] 2968#2968: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:09:55 [error] 2968#2968: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:09:55 [warn] 2976#2976: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:09:55 [error] 2976#2976: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:12:58 [warn] 3004#3004: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:12:58 [error] 3004#3004: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:14:44 [warn] 3013#3013: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:14:44 [error] 3013#3013: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error:02001002:system li
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:20600080:BIO routines:BIO_new_file:no such file)
2025/12/29 21:19:25 [warn] 3039#3039: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:19:38 [warn] 3047#3047: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:19:38 [warn] 3050#3050: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
2025/12/29 21:48:28 [warn] 3212#3212: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/nginx.conf:47
```

Bonus Tasks (10 marks extra credit)

Bonus 1: Custom Error Pages (3 marks)

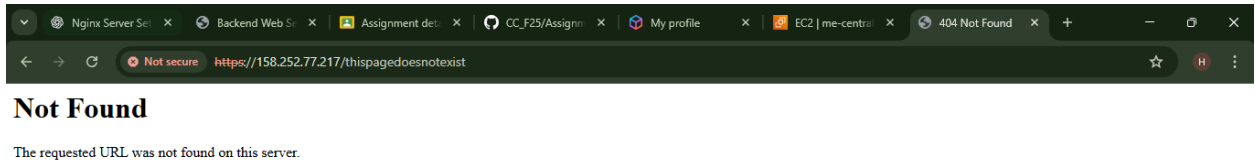
Create custom error pages for Nginx (404, 502, 503).

Tasks:

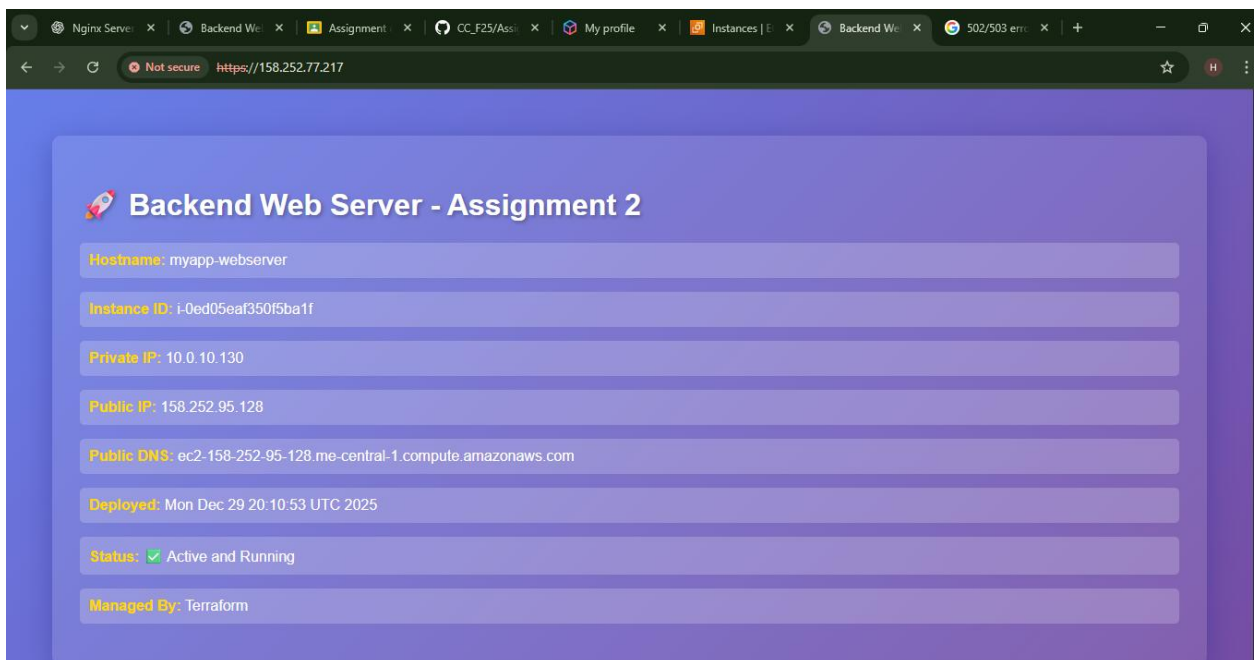
- Create custom HTML error pages
- Configure Nginx to use custom error pages
- Test by accessing non-existent URL and stopping backend servers

Deliverables:

- Screenshot: bonus1_custom_404.png



- Screenshot: bonus1_custom_502.png



Bonus 2: Implement Rate Limiting (3 marks)

Add rate limiting to prevent abuse.

Configuration Example:

```
http {
    limit_req_zone $binary_remote_addr zone=mylimit:10m rate=10r/s;

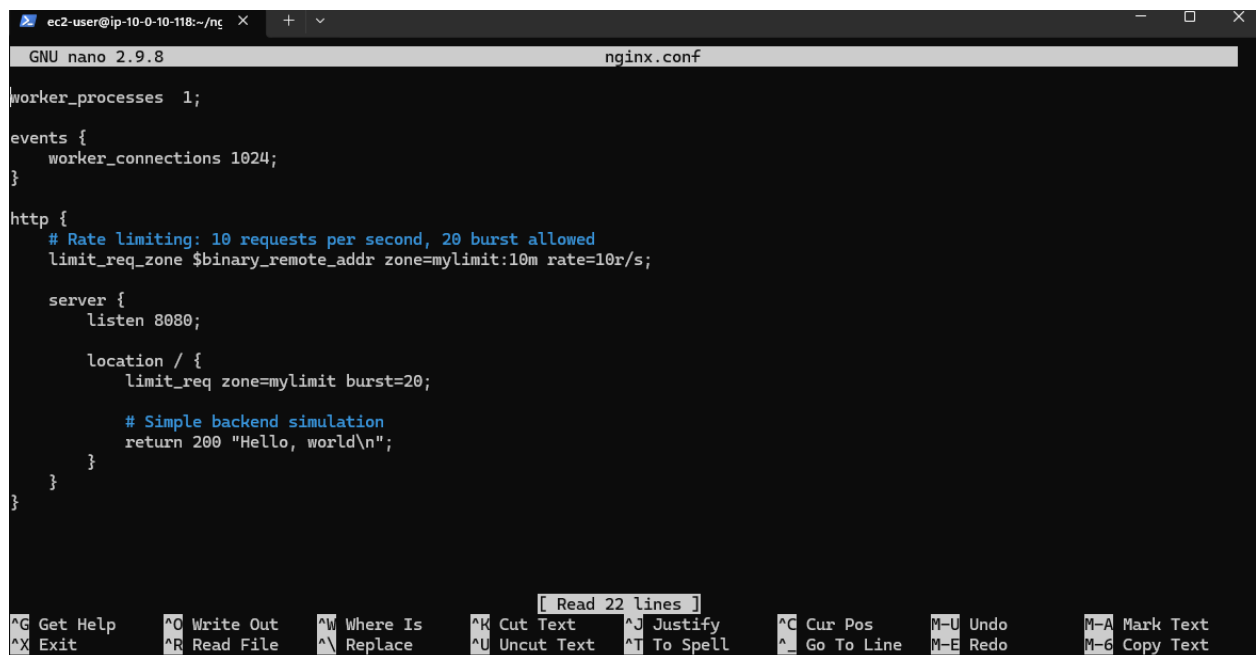
    server {
        location / {
            limit_req zone=mylimit burst=20;
            proxy_pass http://backend_servers;
        }
    }
}
```

Tasks:

- Implement rate limiting
- Test with rapid requests
- Show 429 (Too Many Requests) response

Deliverables:

- Screenshot: bonus2_rate_limit_config.png



```
ec2-user@ip-10-0-10-118:~/ng x + v
GNU nano 2.9.8 nginx.conf

worker_processes 1;

events {
    worker_connections 1024;
}

http {
    # Rate limiting: 10 requests per second, 20 burst allowed
    limit_req_zone $binary_remote_addr zone=mylimit:10m rate=10r/s;

    server {
        listen 8080;

        location / {
            limit_req zone=mylimit burst=20;

            # Simple backend simulation
            return 200 "Hello, world\n";
        }
    }
}
```

- Screenshot: bonus2_rate_limit_test.png


```
Test-NetConnection - 10.0.10.130:80
Attempting TCP connect

Waiting for response

At line:1 char:1
+ mkdir -p ~/health_check
+ ~~~~~
+ CategoryInfo          : ResourceExists: (C:\Users\admin\health_check:String) [New-Item], IOException
+ FullyQualifiedErrorId : DirectoryExist,Microsoft.PowerShell.Commands.NewItemCommand

PS C:\Users\admin> cd ~/health_check
PS C:\Users\admin\health_check> notepad health_check.ps1
PS C:\Users\admin\health_check> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
PS C:\Users\admin\health_check> .\health_check.ps1
ALERT: Server 10.0.10.177 is DOWN
ALERT: Server 10.0.10.146 is DOWN
ALERT: Server 10.0.10.130 is DOWN
ALERT: Server 10.0.10.177 is DOWN
ALERT: Server 10.0.10.146 is DOWN
ALERT: Server 10.0.10.130 is DOWN
ALERT: Server 10.0.10.177 is DOWN
ALERT: Server 10.0.10.146 is DOWN
ALERT: Server 10.0.10.130 is DOWN
ALERT: Server 10.0.10.177 is DOWN
ALERT: Server 10.0.10.146 is DOWN
```

Part 6: Documentation & Cleanup (10 marks)

6.1 README Documentation (5 marks)

Create a comprehensive README.md with:

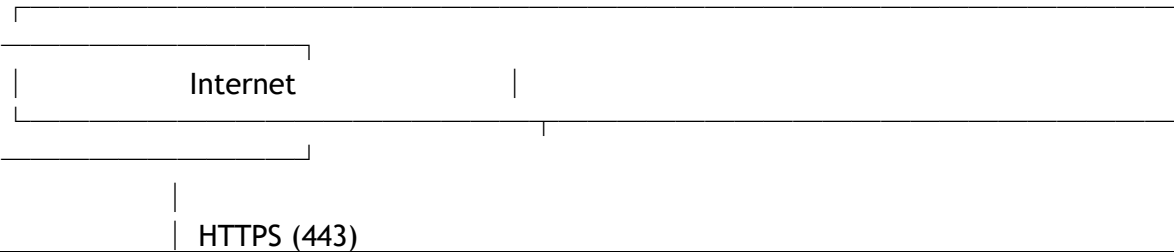
Required Sections:

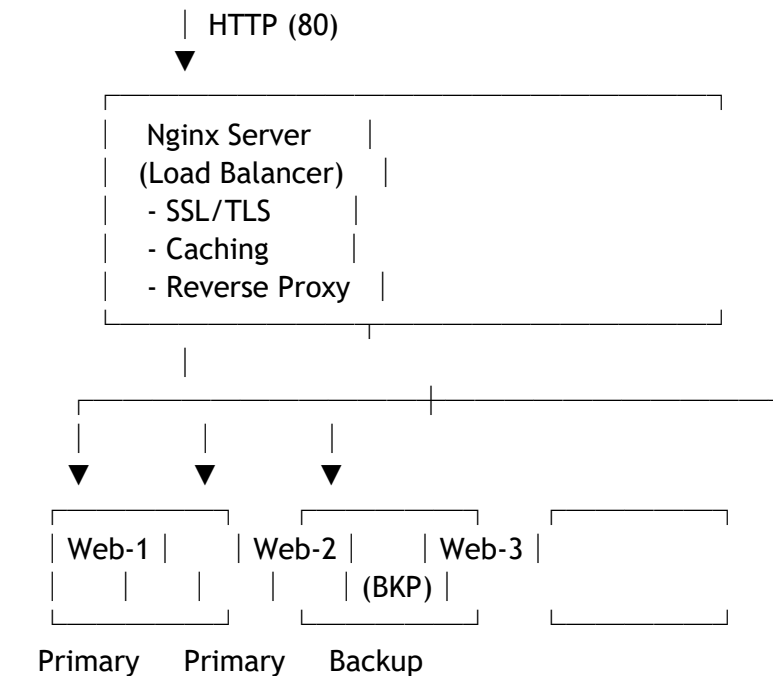
- 1. **Project Overview**
 - o Architecture diagram (can be text-based or image)
 - o Components description
- 2. **Prerequisites**
 - o Required tools
 - o AWS credentials setup
 - o SSH key setup
- 3. **Deployment Instructions**
 - o Step-by-step guide
 - o Variable configuration
 - o Terraform commands
- 4. **Configuration Guide**
 - o How to update backend IPs
 - o Nginx configuration explanation
 - o Testing procedures
- 5. **Architecture Details**
 - o Network topology
 - o Security groups explanation
 - o Load balancing strategy
- 6. **Troubleshooting**
 - o Common issues and solutions
 - o Log locations
 - o Debug commands

Example Structure:

Assignment 2 - Multi-Tier Web Infrastructure

Architecture Overview





Components

Deliverables:

- Screenshot: assignment_part6_readme.png (README.md content)

```

File Edit View H1 H2 H3 B I L A
# Assignment 2 - Multi-Tier Web Infrastructure

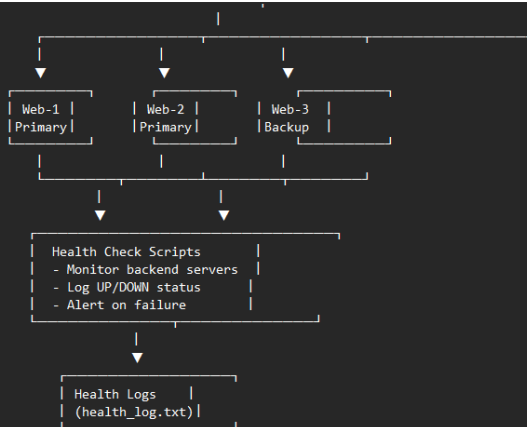
## Project Overview
This project implements a multi-tier web infrastructure on AWS using Terraform and Nginx.
The architecture includes a load-balanced Nginx server with SSL/TLS, caching, and reverse proxy,
serving multiple backend web servers with failover capability.
Architecture Diagram

Add a text-based diagram (or an image if you have one):

## Architecture Diagram

## Architecture Diagram

graph TD
    Internet[Internet] --> Nginx[Nginx Server  
(Load Balancer)  
- SSL/TLS Termination  
- Reverse Proxy  
- Caching  
- Rate Limiting]
    Nginx --> Web1[Web-1]
    Nginx --> Web2[Web-2  
(BKP)]
    Nginx --> Web3[Web-3]
    Web1 --- P1[Primary]
    Web2 --- P2[Primary]
    Web3 --- B[Backup]
  
```



Legend:

- Web-1 & Web-2: Primary backend servers
- Web-3: Backup server
- Nginx: Load balancer handling HTTPS, caching, reverse proxy, and rate limiting
- Health Check Scripts: Monitor server health and log status

C. Components Description

- List main components and their roles:

```markdown

#### ## Components Description

- **Nginx Server**: Load balancer, SSL termination, caching, reverse proxy
- **Web-1 & Web-2**: Primary backend servers
- **Web-3**: Backup server
- **Terraform**: Infrastructure provisioning
- **Health Check Scripts**: Monitors backend servers and logs status

#### D. Prerequisites

List all required tools and credentials:

#### ## Prerequisites

- **Required Tools**:
  - Terraform
  - AWS CLI
  - Nginx
  - PowerShell or Bash (for scripts)
- **AWS Credentials Setup**:
  - Configure AWS CLI with `aws configure`
  - Store Access Key ID and Secret Access Key securely
- **SSH Key Setup**:
  - Generate a key pair for EC2 instances
  - Add the public key in AWS and private key to local machine

#### Deployment Instructions

#### ## Deployment Instructions

- Configure AWS CLI with `aws configure`
- Store Access Key ID and Secret Access Key securely

- **SSH Key Setup**:
  - Generate a key pair for EC2 instances
  - Add the public key in AWS and private key to local machine

#### Deployment Instructions

#### ## Deployment Instructions

1. Configure variables in `variables.tf`
2. Run Terraform commands:
 

```

      ```bash
      D:\terraform_1.14.3_windows_amd64\terraform.exe init
      D:\terraform_1.14.3_windows_amd64\terraform.exe plan
      D:\terraform_1.14.3_windows_amd64\terraform.exe apply -auto-approve
      D:\terraform_1.14.3_windows_amd64\terraform.exe validate
      
```

Verify backend IPs and Nginx config

Ensure health check scripts are running

**F. Configuration Guide

```markdown

#### ## Configuration Guide

- **Updating Backend IPs**:
  - Edit the upstream block in Nginx config
  - Update `BACKENDS` array in health check scripts

- **Nginx Configuration Explanation**:

## 6.2 Infrastructure Cleanup (5 marks)

Properly destroy all resources and verify cleanup.

### Tasks:

- Run terraform destroy
- Confirm resource deletion in AWS Console
- Verify no remaining resources
- Document the destruction process

### Commands:

```
Destroy infrastructure
terraform destroy
```

```
Type 'yes' when prompted
```

```
Verify state file is empty
cat terraform.tfstate
```

```
List any remaining resources (should be empty)
```

```
aws ec2 describe-instances --filters "Name=tag:Project,Values=Assignment-2" --query
"Reservations[]. Instances[]. InstanceId"
```

### Deliverables:

- Screenshot: assignment\_part6\_terraform\_destroy\_prompt.png

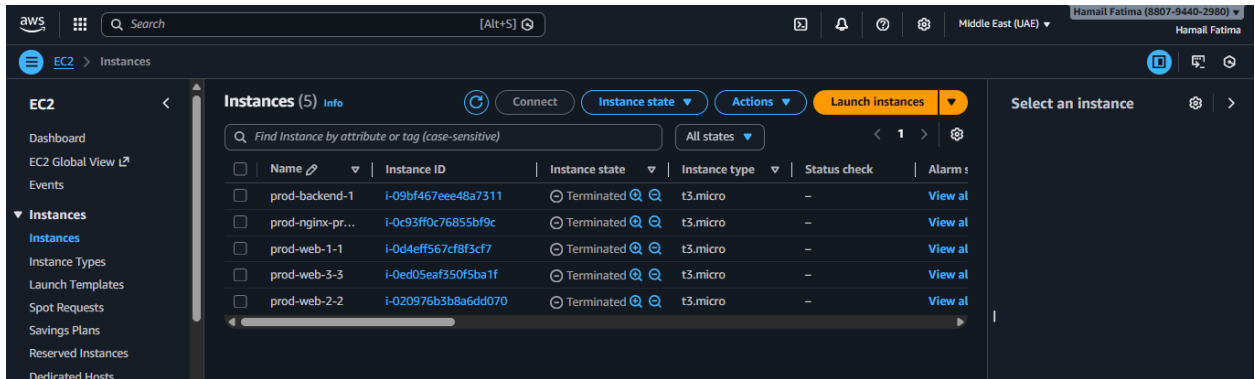
```
PS D:\Assignment2> D:\terraform_1.14.3_windows_amd64\terraform.exe destroy
Acquiring state lock. This may take a few moments...
data.http.my_ip: Reading...
module.security.data.http.my_ip: Reading...
module.backend_servers["web-3"].data.aws_ami.amazon_linux: Reading...
module.backend_servers["web-2"].data.aws_ami.amazon_linux: Reading...
module.nginx_server.data.aws_ami.amazon_linux: Reading...
module.backend_server_1.data.aws_ami.amazon_linux: Reading...
module.backend_servers["web-1"].data.aws_ami.amazon_linux: Reading...
module.networking.aws_vpc.this: Refreshing state... [id=vpc-060128023c9a23264]
aws_key_pair.terraform_key: Refreshing state... [id=terraform-key]
module.security.data.http.my_ip: Read complete after 1s [id=https://icanhazip.com]
data.http.my_ip: Read complete after 1s [id=https://icanhazip.com]
module.backend_servers["web-2"].data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.nginx_server.data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.backend_server_1.data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.backend_servers["web-3"].data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.backend_servers["web-1"].data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.networking.aws_internet_gateway.this: Refreshing state... [id=igw-00c418b7e77189b31]
module.networking.aws_subnet.this: Refreshing state... [id=subnet-04cacd3f78376e5a3]
module.security.aws_security_group.nginx_sg: Refreshing state... [id=sg-0addbdc83a4f35e04]
module.networking.aws_route_table.this: Refreshing state... [id=rtb-02bdb69fefe0a0d0e]
module.security.aws_security_group.backend_sg: Refreshing state... [id=sg-08e027917340b1443]
module.nginx_server.aws_instance.this: Refreshing state... [id=i-0c93ff0c76855bf9c]
module.networking.aws_route_table_association.this: Refreshing state... [id=rtbassoc-088ef56c30ae320a5]
module.backend_server_1.aws_instance.this: Refreshing state... [id=i-09bf467eee48a7311]
module.backend_servers["web-3"].aws_instance.this: Refreshing state... [id=i-0ed05eaf350f5ba1f]
```

- Screenshot: assignment\_part6\_terraform\_destroy\_complete.png

```
module.security.aws_security_group.nginx_sg: Destruction complete after 0s
module.networking.aws_vpc.this: Destroying... [id=vpc-060128023c9a23264]
module.networking.aws_vpc.this: Destruction complete after 0s

Destroy complete! Resources: 13 destroyed.
PS D:\Assignment2> |
```

- Screenshot: assignment\_part6\_aws\_instances\_destroyed.png (AWS Console showing no instances)



- Screenshot: assignment\_part6\_empty\_state. png (empty terraform.tfstate)

```
PS D:\Assignment2> cat terraform.tfstate
{
 "version": 4,
 "terraform_version": "1.14.3",
 "serial": 105,
 "lineage": "c853e6b2-65e3-4697-a17a-1a98ead9dfeb",
 "outputs": {},
 "resources": [],
 "check_results": [
 {
 "object_kind": "var",
 "config_addr": "var.subnet_cidr_block",
 "status": "unknown",
 "objects": null
 },
 {
 "object_kind": "var",
 "config_addr": "var.vpc_cidr_block",
 "status": "unknown",
 "objects": null
 }
]
}
```

```
PS D:\Assignment2> aws ec2 describe-instances --filters "Name=tag:Project,Values=Assignment-2" --query "Reservations[].Instances[].InstanceId"
Traceback (most recent call last):
 File "PyInstaller\hooks\rthooks\pyi_rth_multiprocessing.py", line 54, in <module>
 File "PyInstaller\hooks\rthooks\pyi_rth_multiprocessing.py", line 16, in _pyi_rthook
 File "PyInstaller\loader\pyimod02_importers.py", line 384, in exec_module
 File "multiprocessing__init__.py", line 16, in <module>
 File "PyInstaller\loader\pyimod02_importers.py", line 384, in exec_module
 File "multiprocessing\context.py", line 6, in <module>
 File "PyInstaller\loader\pyimod02_importers.py", line 384, in exec_module
 File "multiprocessing\reduction.py", line 16, in <module>
 File "PyInstaller\loader\pyimod02_importers.py", line 384, in exec_module
 File "socket.py", line 52, in <module>
ModuleNotFoundError: No module named '_socket'
[PYI-18920:ERROR] Failed to execute script 'pyi_rth_multiprocessing' due to unhandled exception!
PS D:\Assignment2>
```

